NEUROSHELL INTENT RECOGNITION ENGINE (IRE)

P. M. T. L. Karunarathna
(IT22249852)

Dissertation submitted in partial fulfillment of the requirements
for the Bachelor of Science (Hons) in Information Technology

Department of Information Technology
Sri Lanka Institute of Information Technology
Sri Lanka

August 2026

DECLARATION OF THE CANDIDATE & SUPERVISOR

I declare that this is my own work and this dissertation does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the nonexclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

P. M. T. L. Karunarathna (IT22249852)

Signature: _____ Date: _____

Statement of the supervisor

The above candidate has carried out research for the bachelor's degree dissertation under my supervision.

Signature of the supervisor: _____ Date: _____

[TODO: Supervisor Name]

Abstract

Large language models (LLMs) are increasingly used to translate natural-language security requests into machine-executable tool actions, but their stochastic output—malformed schemas, hallucinated parameters, out-of-scope targets and adversarial phrasing—makes them unsafe to feed directly to automated security tooling. This dissertation designs, implements and evaluates NeuroShell IRE, a guarded intent-recognition engine that constrains the raw output of a small, offline, CPU-only language model with deterministic validation before any downstream dispatch.

The methodology is a controlled design-and-evaluation study under a committed protocol: a purpose-built synthetic data set of 1,500 security prompts (600 well-formed, 900 flawed across eight flaw classes) is scored through a seven-stage pipeline wrapped in five middleware components, using two offline backends (Gemma and Qwen) under four experiments measuring pipeline value, backend head-to-head, per-layer ablation and per-class catch. On the evaluated records, the guarded pipeline raises decision accuracy from 69–78% to 82–86%, raises the flaw-catch rate from 0% to 64–72%, and lowers the false-positive rate from 31–32% to 14–15%. Ablations show the deterministic validation stack is the dominant safety contributor: removing it reduces the catch rate by 30 and 24 points respectively, and removing the hallucination validator alone by 9 and 14 points; the catch-rate ordering is reproduced across both backends.

The dissertation concludes that a small offline model can be made reliable enough to gate an automated security copilot provided the trust boundary remains closed (retrieval-free, with no free-form output rendered downstream). Because the evaluation consists of single-run point estimates on a synthetic, non-uniform data set on a single device, external validity is bounded. Future work should remove the single-run limitation, strengthen the weakly protected semantic flaw classes, and close the verified engineering gaps—default development credentials and the inactive `production` scope mode—before any deployment.

Keywords: Intent Recognition, LLM Guardrails, Hallucination Detection, Offline Language Models, Security Copilots

ACKNOWLEDGEMENT

[TODO: Thank staff, fellow students and external bodies who helped with materials, data, design, analysis and writing; name the sponsor/funding organisation if any; acknowledge the supervisor.]

TABLE OF CONTENTS

Declaration of the Candidate & Supervisor i

Abstract ii

Acknowledgement iii

Table of Contents iv

List of Figures vii

List of Tables viii

List of Abbreviations ix

List of Appendices x

1. Introduction 1

    1.1 Overall Project Context 1

    1.2 Individual Component Context 1

    1.3 Problem Statement 2

    1.4 Research Gap 3

    1.5 Aim and Objectives 4

    1.6 Research Questions 5

    1.7 Scope, Limitations and Assumptions 5

    1.8 Individual Contribution 6

    1.9 Thesis Structure 7

2. Literature Review and Related Work 9

    2.1 Review Method and Scope 9

    2.2 Domain and Theoretical Foundations 10

    2.3 Related Research 13

    2.4 Existing Systems, Products and Techniques 16

    2.5 Gap Synthesis and Conceptual Framework 19

3. Research Methodology 22

    3.1 Research Design 22

3.2 Data and Study Materials 25
3.3 Experimental Procedure 27
3.4 Measures and Evaluation Metrics 29
3.5 Data Analysis 31
3.6 Ethics, Privacy, Security and Responsible AI 33
3.7 Reproducibility and Research Artefacts 35
4. Product and Solution Design 38
4.1 Requirements 38
4.2 Component Architecture within the Complete Solution 40
4.3 Design Alternatives and Decisions 40
4.4 Detailed Design 42
4.5 Standards and Professional Practice 44
5. Implementation, Integration and Testing 47
5.1 Development Environment and Technology Stack 47
5.2 Component Implementation 48
5.3 Integration with the Other Components 50
5.4 Verification and Software Testing 51
5.5 Deployment and Operation 52
6. Results and Validation 54
6.1 Research Results 54
6.1.1 Experiment 1: Pipeline Value (RQ1) 54
6.1.2 Experiment 2: Backend Head-to-Head (RQ3) 55
6.1.3 Experiment 3: Layer Ablation (RQ2 and RQ3) 55
6.1.4 Experiment 4: Per-Class Catch (RQ2) 55
6.2 Product and Technical Results 57
6.2.1 Verification and testing 57
6.2.2 Known limitation achieved 57
6.3 User or Stakeholder Validation 57

6.4 Integration Results 57
6.5 Summary of Findings 58
7. Discussion 59
7.1 Interpretation of Findings 59
7.2 Comparison with Literature Baselines and Existing Solutions 60
7.3 Research Contribution 61
7.4 Product Contribution 62
7.5 Limitations and Threats to Validity 63
7.6 Implications 64
8. Commercialisation and Sustainability 66
8.1 Value Proposition and Target Users 66
8.2 Market and Competitive Position 67
8.3 Business and Sustainability Model 68
8.4 Scalability, Intellectual Property and Compliance 69
8.5 Contribution to the Sustainable Development Goals 70
9. Conclusion and Future Directions 72
9.1 Conclusion 72
9.2 Achievement of Objectives 73
9.3 Individual Research and Product Contributions 74
9.4 Future Directions 74
Reference List 76
Glossary 78
Appendix A: Research Instruments or Extended Results 79
Appendix B: Academic Integrity and AI Use Disclosure 80

LIST OF FIGURES

Page

LIST OF TABLES

Page

Table 2.1 Surveyed existing systems and techniques against the properties adopted in this dissertation. Judgements summarise the cited primary sources and are re-verified at the final AUDIT. 18

Table 3.1 Experimental data set composition ($N = 1500$). D^+ holds 600 well-formed requests across the nine-intent taxonomy; D^- holds 900 deliberately flawed requests across eight flaw categories. 26

Table 4.1 Role model for the IRE. The default role is analyst; unknown roles coerce to viewer (least privilege). 38

Table 4.2 Verified pipeline stages and middleware under `src/`. 46

Table 5.1 Verified technology stack. 47

Table 6.1 Experiment 1: raw model output against the guarded pipeline on the full 1500-record set (N=1500, D^+=600, D^-=900). 54

Table 6.2 Experiment 2: identical guardrail stack, both backends. 55

Table 6.3 Experiment 3: ablation on the full 1500-record set. ΔCatch is in percentage points relative to the fully guarded pipeline (a negative value means the layer contributes); Noise is the approximate run-to-run variation in percentage points. The validation-stack row is the joint S5–S7 ablation. 56

Table 6.4 Experiment 4: per-class catch rate and primary detection mechanism (deep-dive subset, each class's own denominator). 56

LIST OF ABBREVIATIONS

Abbreviation	Description
API	Application Programming Interface
CPU	Central Processing Unit
CVE	Common Vulnerabilities and Exposures
HTTP	Hypertext Transfer Protocol
IRE	Intent Recognition Engine
JSON	JavaScript Object Notation
LLM	Large Language Model
NLU	Natural Language Understanding
RBAC	Role-Based Access Control

[TODO: Reconcile this list with the final text: every listed abbreviation must occur at least once in the thesis and every abbreviation used must be listed (M7).]

LIST OF APPENDICES

Appendix	Description	Page
Appendix A	Research Instruments or Extended Results	79
Appendix B	Academic Integrity and AI Use Disclosure	80

1. INTRODUCTION

1.1 Overall Project Context

Modern security operations depend on specialised tooling to discover, assess and remediate faults across networks and applications. As the scale of this work grows, the manual translation of an operator's intent into safe, well-scoped tool actions becomes a significant bottleneck, and mistakes made at this point—an incorrect target, a malformed instruction, or an out-of-scope action—can affect production infrastructure. Large Language Models (LLMs) are attractive for this translation step because they can interpret natural language and emit structured, machine-readable instructions that drive security tooling [1]. Yet a model is a stochastic component: its output may be malformed, misclassified, hallucinated, or the product of injected instructions, and a security pipeline that acts on such output without further checking transfers every one of these errors into the actions it executes.

NeuroShell is an autonomous, natural-language-driven security orchestration framework developed as a four-component research chain: the Intent Recognition Engine (IRE), a Dynamic Planner, an Adaptive Execution and Error Recovery engine, and an AI-driven Vulnerability Analysis engine (project reference R26-CS-018). The framework is designed to let an analyst express offline security tasks in natural language and have the resulting validated, structured contracts drive downstream automation, within an explicitly configured engagement scope. It therefore sits at the intersection of cybersecurity automation, software engineering and applied machine learning. Its central concern is the first step of that chain: the point at which human intent is converted into a machine-executable form, because an error introduced there propagates through every stage that follows and must be detected before it takes effect.

This dissertation concentrates on a single component of the framework and asks whether, in an offline CPU-only deployment, a small instruction-tuned language model can be made sufficiently reliable to serve as the head of an automated security pipeline when its raw output is constrained by deterministic validation.

1.2 Individual Component Context

The Intent Recognition Engine (IRE) is the first component of the chain (Component 01). It acts as a domain-specific Natural Language Understanding (NLU) gateway: it receives natural-language security commands—for example a request to perform a

targeted network discovery scan—and returns a structured, validated intent contract that downstream components consume without re-interpreting prose.

The component boundary is deliberately narrow. The IRE is concerned with understanding and validating *intent*; it does not decide how a task is scheduled, across which hosts it runs, or how execution failures are recovered. Those responsibilities belong to the Dynamic Planner (Component 02) and to the Adaptive Execution and Error Recovery engine (Component 03), which consume the IRE's output. The handoff is designed to be safe by construction: the output contract carries an explicit intent type, a resolved target, the recognised options and any validation findings, and the downstream contract requires that actions derived for rejected intents must not be executed.

Because the IRE sits at the head of the chain, its reliability bounds the reliability of the components that follow. A target that is mislabeled, an intent that is misclassified, or a hallucinated identifier that is accepted would be inherited by every downstream action. The component therefore couples an instruction-tuned language model with deterministic validation and guard stages whose purpose is to contain model error before it can propagate to a planner or executor. This dissertation evaluates whether that guarded design achieves sufficiently accurate and error-resilient intent recognition to justify acting on model-produced security instructions in a controlled, offline setting.

1.3 Problem Statement

Security operations are bottlenecked by the manual translation of operator intent into tool actions. A penetration tester or analyst must map free-form instructions—"scan the internal network for EternalBlue", "check this host for CVE-2021-44228"—onto commands, targets and constraints while respecting authorisation boundaries. Recent work shows that Large Language Models (LLMs) can act as capable copilots for parts of this work, producing realistic attack plans and driving tools in controlled environments [1, 2]. Using an LLM for this translation step is attractive precisely because natural language is the operator's native interface.

The problem is that a language model is not a reliable translator. Its output can be malformed, misclassified, or wholly fabricated: LLMs hallucinate identifiers and parameters [3], and they can be diverted by adversarially crafted utterances [4, 5]. In a purely conversational copilot these errors are display text that a human can vet. In an *automated* security pipeline, where intent is converted directly into actions, each such error becomes a candidate machine-dispatched action: a fabricated CVE is looked up, a wrong target is scanned, or an injected instruction is obeyed. Acting on unvalidated

model output therefore transfers every residual error of the model into the security toolchain it drives.

The intent-recognition step concentrates this risk. A copilot that pipelines raw LLM output into a planner and executor must, at minimum, emit a well-formed, schema-conformant, authorised intent structure before any downstream component acts on it. Unless that structured output is validated deterministically, the entire framework inherits the model's failure modes. This dissertation addresses the resulting problem: *how can a small, offline, CPU-only language model be made sufficiently reliable to serve as the intent-recognition head of an automated security pipeline, when its raw output is constrained by deterministic validation rather than by a human reviewer or a cloud back-end?*

1.4 Research Gap

Three related gaps motivate this work.

First, LLM copilots for security tooling validate their output informally, if at all. PENTESTGPT decomposes penetration-testing tasks across a reasoning, generation and guidance chain and does not subject its output to deterministic schema, scope or authorisation validation before machine consumption [1]; the sparring-partner systems demonstrated by Happe and Cito produce plausible next actions consumed by a human [2]. When a copilot's output is fed to an *automated* toolchain—the natural integration point for intent recognition—the requirement changes from plausibility to correctness: a hallucinated CVE, malformed structure, or injected text becomes a machine-dispatched action rather than display text. The OWASP Top 10 for LLM applications catalogues prompt injection, insecure output handling and related risks as first-class application concerns [6], yet the systematic defence of machine-consumed intent remains largely unaddressed in the surveyed copilot work.

Second, the defence literature either assumes interactive, cloud-hosted generation or takes the retrieval-augmented route. Sampling-based methods for detecting fabricated output, such as SelfCheckGPT [7], semantic uncertainty [8] and semantic entropy [9], and the self-consistency decoding strategy [10], all assume repeated or interactive generation with generous compute. Retrieval-augmented generation [11] grounds answers externally, but introduces retrieved data as a prompt-injection surface [4] and requires corpora and embeddings infrastructure at inference time. The operational setting targeted here—a small offline model running unattended on an operator's laptop,

with latency budget that rules out a retry loop or a verifier LLM—is therefore not covered by direct prior measurements.

Third, to the best of the review conducted here, no existing autonomous penetration-testing framework ships a dedicated, isolated natural-language understanding layer that reliably converts free-form offensive-security instructions into validated, schema-constrained machine contracts; tools either accept structured input or rely on general-purpose LLMs without domain-specific validation. Leaving this step to a general chat model without a purpose-built validation layer is exactly the unsafe pattern identified above. The research gap this dissertation fills is therefore the design and *measurement* of a guarded, retrieval-free intent recognition engine—an isolated NLU component with deterministic validation— for offline, single-device security deployments.

1.5 Aim and Objectives

The aim of this dissertation is to design, implement and evaluate a guarded intent-recognition engine that constrains the raw output of a small, offline, instruction-tuned language model with deterministic validation, so that the combined system can act as a reliable head of an automated security pipeline in a CPU-only, single-device deployment. To achieve this aim the following objectives are pursued:

1. To design and implement the Intent Recognition Engine (IRE): a component that receives free-text security requests and emits strictly schema-conformant, authorised intent contracts, consuming a small offline language model as a proposition generator whose output is validated deterministically before it is handed onward.
2. To construct a purpose-built evaluation data set of security prompts, covering well-formed requests and deliberately flawed prompts across the flaw categories that matter in practice (adversarial phrasing, out-of-scope targets, ambiguous or contradictory requests, fabricated parameters and identifiers, and role violations), each annotated with gold labels.
3. To establish an evaluation protocol that quantifies, on the committed experiment record, how much the guarded pipeline improves structured intent recognition and flaw catching over raw model output, and that abates each guard layer independently to estimate its individual contribution.
4. To compare two offline language-model backends under the identical guardrail stack, in order to measure how much of the achieved safety is backend-independent rather than an artefact of one model.

Each objective is addressed by a corresponding chapter of this dissertation: the design is developed in Chapter 4, the data set and protocol in Chapter 3, the implementation in Chapter 5, and the evaluation and backend comparison in Chapter 6.

1.6 Research Questions

The central question guiding this dissertation is whether a small, offline, unattended language model can be made reliable enough to drive an automated security copilot once its output is subject to deterministic validation. From this central question three concrete research questions are derived:

1. **RQ1 (pipeline value).** How much does a stacked validation pipeline improve structured intent-recognition accuracy and flaw catching over raw language-model output?
2. **RQ2 (layer contribution).** Which individual guard layer contributes the most to the flaw-catch rate, and which layers interact destructively when combined?
3. **RQ3 (generalisability).** Do the guardrails generalise across two different offline language-model backends, or is the measured safety an artefact of a single model?

The study is conducted as an empirical, measurement-based investigation rather than a hypothesis-testing one: the outcomes of RQ1–RQ3 are quantified from the committed experiment record reported in Chapter 6, and no statistical significance claim is attached to the single-run measurements (see Section 1.7).

1.7 Scope, Limitations and Assumptions

Scope

This dissertation is an individual study of a single component, the Intent Recognition Engine (Component 01 of the NeuroShell framework, project reference R26-CS-018), and does not implement the downstream planner, executor or vulnerability-analysis components. Within that boundary, the scope covers: (1) a guarded NLU pipeline that accepts free-text security requests and emits validated intent contracts; (2) the deterministic validation and security countermeasures that constrain the model's output; (3) an offline, CPU-only, single-device deployment in which nothing is retrieved at inference time and no request leaves the device; and (4) a closed intent taxonomy and an operator-configured engagement scope that bounds permitted targets and techniques.

Evaluation is conducted on a curated, annotated data set of prompts in a lab harness, not against live production traffic.

Limitations

The principal limitation is evidential: the results reported in this dissertation rest on a bounded, committed experiment record of single-run point estimates, without confidence intervals or seed sweeps, and therefore carry residual uncertainty that is discussed in Chapter 7. Only two offline language-model backends are compared under the guardrail stack, and the evaluation data set is author-authored rather than drawn from observed operational traffic. Measurements are specific to the evaluation hardware and CPU-only inference runtime, so latency and throughput figures are indicative rather than generalisable. The framework-level benefit of the IRE is validated at the component level; end-to-end orchestration with the downstream components is outside the measured evidence presented here.

Assumptions

The following assumptions underpin the design and evaluation:

- The operator configures the engagement scope and authorisation policy correctly; the scope guard can only enforce the boundary it is given.
- The deployment device and the downloaded model weights are trusted; physical compromise of the device and backdoors embedded in the model weights are out of scope.
- Downstream components consume the intent contract as specified and honour the contract rule that actions derived for rejected intents must not be executed.
- Because the pipeline performs no retrieval and renders no free-form output, indirect prompt injection via retrieved content and tool injection via unfiltered rendered text are structurally out of scope, narrowing the attack surface to the classification of the single user utterance.
- The annotated prompts in the evaluation data set are representative of the phrasing an operator would use with the system.

1.8 Individual Contribution

This dissertation is an individual thesis on Component 01 of the NeuroShell four-component research chain. The candidate's individual contribution is the design,

implementation and evaluation of the Intent Recognition Engine, and comprises: (1) the architecture of the guarded NLU gateway, in which a small offline language model acts as a proposition generator and deterministic stages validate structure, target syntax and authorisation before an intent is emitted; (2) the security countermeasures layered around the core—including the authorisation checks, the adversarial-utterance detection, and the self-consistency and hallucination-validation gate—that enforce the safety invariant that every accepted intent is schema-valid, authorised and in scope; (3) the authoring of the annotated evaluation data set covering well-formed and deliberately flawed prompt categories; (4) the evaluation harness and the per-layer ablation protocol used to quantify the pipeline's value and each layer's contribution; and (5) the empirical comparison of two offline language-model backends under the identical guardrail stack.

The other components of the NeuroShell chain—the dynamic planner, the adaptive executor, and the vulnerability-analysis engine—provide the context in which the IRE operates and are described only to the extent needed to define the IRE's interface contract. The candidate takes responsibility for the correctness of the IRE implementation and for every result reported in this dissertation.

1.9 Thesis Structure

This dissertation is organised as follows. *Chapter 1* (the present chapter) establishes the problem, the research gap, the aim, objectives, research questions and scope of the study. *Chapter 2* reviews related work on LLM copilots for security, adversarial attacks on LLM-integrated applications, and hallucination-detection and uncertainty methods, and situates the guarded intent-recognition approach within it. *Chapter 3* presents the research approach: the objectives-to-method mapping, the design of the evaluation data set, the metrics, and the experiment and ablation protocol used to answer RQ1–RQ3. *Chapter 4* develops the product and solution design of the Intent Recognition Engine, including the pipeline stages, the security countermeasures and the intent-contract interface. *Chapter 5* describes the implementation, the integration of the two offline backends, and the automated test suite. *Chapter 6* reports the results of the pipeline-value, layer-ablation and backend-comparison experiments and interprets them against the research questions. *Chapter 7* discusses the findings, the threats to validity, and the residual failure modes of the guarded design. *Chapter 8* considers the commercialisation and sustainability of the approach. *Chapter 9* concludes the dissertation, restates the contributions, and outlines directions for future work. Supplementary material appears in the appendices: the data-set and evaluation instruments (Appendix A)

and the declaration on academic integrity and the use of artificial-intelligence tools (Appendix B).

2. LITERATURE REVIEW AND RELATED WORK

2.1 Review Method and Scope

This chapter reviews the literature needed to support the research questions posed in Section 1.6. The review is deliberately targeted rather than bibliometric: its purpose is to establish (1) how language-model copilots for security operations behave and how they validate their own output; (2) the attack surface of applications that integrate language models with tooling; (3) the available methods for detecting model fabrication and for measuring answer agreement or uncertainty; and (4) which of these techniques remain viable under the deployment constraints adopted in this dissertation—an offline, CPU-only, single-inference budget with no retrieval at query time. These four questions map directly onto RQ1–RQ3.

The corpus is drawn from the computing literature and from authoritative technical sources. Peer-reviewed and established-venue material was sought across the ACM proceedings (including the ESEC/FSE joint conference and the Artificial Intelligence and Security (AISec) workshop), the empirical NLP and machine learning conferences (EMNLP, ICLR, NeurIPS), the ACM Computing Surveys journal, and the arXiv preprint archive; standards and tooling references were taken from their canonical sources (the IETF RFC series, the OWASP project, and the official documentation of the Nmap and Metasploit tools). Candidate works were located with controlled query themes covering the joint areas of interest: security copilots and penetration testing assisted by language models, prompt injection and indirect prompt injection, hallucination detection, semantic uncertainty, self-consistent decoding, retrieval-augmented generation, and authorisation control for tool-driven systems.

Works were included when they satisfied two criteria: relevance to at least one research question, and a verifiable primary source reporting either an empirical evaluation or an authoritative specification. The machine-learning literature covered is concentrated in the period 2019–2025, which spans the advent of the sentence-embedding and language-model techniques relied on here [11, 12]; older technical standards and tooling references are included regardless of year because they define stable interfaces rather than evolving research claims. Preprint versions were preferred only when a peer-reviewed version was not available for a work; each entry in the reference list was verified against its real source before it was cited in this dissertation. The review is bounded by the deployment constraint: methods that assume interactive generation, a cloud-hosted verifier, or retrieval infrastructure are surveyed because they are the

competing alternatives against which the guarded pipeline must be justified, and their unsuitability for the offline single-inference budget is part of the analysis in Section 2.5.

The remainder of the chapter is organised as follows. Section 2.2 establishes the theoretical background of intent recognition and of the guarantees expected from a safety-critical interface. Section 2.3 examines the related research on LLM copilots, attacks, and fabrication detection in detail. Section 2.4 reviews existing systems, products and techniques against the same criteria. Section 2.5 synthesises the findings into the gap that motivates this dissertation and into the conceptual framework that the design chapters build on.

[TODO: Confirm whether the faculty's assessment rubric requires a formal systematic-review protocol (screening counts, PRISMA-style reporting). The current review is a targeted narrative review; screening statistics are not reported because no formal screening process was conducted.]

2.2 Domain and Theoretical Foundations

The security-operations domain

The domain this dissertation addresses is offensive and defensive security operations: the systematic discovery, assessment and (where authorised) remediation of weaknesses in networks and applications. The operator's work is organised around a small set of recurring intents—discovering what is exposed, enumerating services, identifying known vulnerabilities, and checking whether specific weaknesses apply to particular targets. These intents map onto specialised tooling: network discovery and scanning (of which Nmap is the canonical example [13]), and exploitation framework drivers (of which Metasploit is the canonical example [14]). Two properties of this domain matter for the work in this dissertation. First, each intent carries a *target* and a set of *parameters*, so the gap between prose and machine dispatch is structural rather than cosmetic. Second, every action is bounded by authorisation: a scan may only touch targets and techniques within an explicitly declared engagement scope, and some intents are reserved for roles with the required mandate. The translation layer evaluated in this dissertation is exactly the component that must preserve both the structure and the authorisation of operator intent.

Intent recognition and natural-language understanding

At its core, intent recognition is a natural-language understanding (NLU) problem: given an utterance, determine the communicative intent and extract the arguments that parameterise it. In a classical NLU system this is split into an intent-classification step, which assigns the utterance to one of a fixed set of intent classes, and a slot-filling (or entity-extraction) step, which recovers typed argument values such as hosts, networks and ports. The intent contract introduced in Section 1.2 is the machine-consumable record of these two steps: a resolved intent type, a target and recognised options, together with any validation findings. For this record to be useful to downstream automation it must be expressed in a schema-bounded, machine-parseable syntax, which is why the IRE emits a structured contract rather than prose.

Representational infrastructure underpins the more advanced parts of the design. Sentence embeddings map natural-language strings into a vector space in which semantically similar strings are close under a distance such as cosine similarity; Sentence-BERT is the representative technique [12]. These embeddings provide the two similarity signals used in this work: the agreement signal by which multiple model outputs are judged to express the same meaning, and the cache signal by which a repeated intent is recognised without re-invoking the model.

Language models as stochastic components

Instruction-tuned language models are the generative core of the IRE. Their defining property for this dissertation is that inference is a sampling process: given a prompt, the model defines a probability distribution over output sequences, and a response is drawn from that distribution. Sampling temperature controls how peaked the distribution is, so the same prompt can yield different outputs across calls. A single response is therefore a *sample* of the model's belief, not a deterministic verdict. This observation is the theoretical basis of the design's treatment of the model as an unreliable proposition generator whose output must be checked rather than trusted. It also underlies consistency-based confidence: the self-consistency decoding strategy samples several responses and strengthens confidence in answers that agree with one another [10], applying a voting intuition that originates in chain-of-thought reasoning and transfers naturally to classification agreement.

Hallucination as a failure class

A second property of language models is that their outputs can be fabricated relative to the true state of the world. The hallucinations relevant here are the *extrinsic* form in the taxonomy of Ji *et al.*: statements that contradict or are unverifiable against an authoritative source, rather than mere inconsistency with an input context [3]. Fabricated security identifiers are a concrete instance: a model may emit a CVE identifier, an IP address or a port that does not exist or is not the one the operator intended. Because a downstream tool will act on those values, the practical requirement is not simply to detect fabrication in general but to detect it in tightly constrained value spaces (identifier formats, address syntaxes, port ranges), where deterministic checking is feasible.

Agreement and uncertainty as fabrication signals

Detection without an external verifier relies on the statistical structure of the model's own output. Sampling-based methods compute a consistency score from a small set of generated responses—SelfCheckGPT scores sampled responses for mutual consistency [7]—while information-theoretic approaches frame the uncertainty of free-form answers through semantic entropy over meaning-equivalence clusters [8, 9]. These methods provide the theoretical grounding for the self-consistency gate in the IRE design. Their common cost, however, is repeated generation: every extra sample multiplies inference time, which collides with the CPU-only, single-user latency budget adopted in this dissertation. That tension—between the statistical value of sampling and the hard cost of each sample—is a central theme returned to in the synthesis (Section 2.5).

Authorisation and network boundaries

Finally, two security foundations bound what the validated contract is allowed to assert. The first is role-based access control (RBAC), under which a user's authorisation to perform an action is determined by the roles assigned to that user and an access matrix that grants or denies the intent for each role; the IRE's four-role model operationalises this standard pattern. The second is network-level scope: private address allocation is codified by RFC 1918 [15], which defines the address ranges reserved for private internets and provides the technical basis for distinguishing an internal target from an external one. Together with the operator-declared engagement scope, these foundations define the predicates that deterministic validation can and cannot decide: syntax, format and authorisation are decidable propositions; only meaning is delegated to the model.

This division of responsibility is the theoretical spine of the solution designed in Chapter 4.

[TODO: If the assessment rubric requires canonical textbook references for NLU fundamentals and for role-based access control, add formal citations for the intent-classification/slot-filling formulation and for the RBAC reference model; none are currently in the reference database.]

2.3 Related Research

This section reviews the three bodies of work most directly related to the Intent Recognition Engine—language-model copilots for security operations, attacks on applications that integrate language models, and methods for detecting model fabrication—followed by a brief assessment of what each body contributes to and leaves open for this dissertation.

Language-model copilots for security operations

Two systems define the current shape of LLM-assisted offensive security. Happe and Cito proposed language models as *sparring partners* for penetration testing and demonstrated a closed-loop system in which a model drove a deliberately vulnerable virtual machine to root access [2]. PENTESTGPT treats penetration testing as a task for a language-model-centric assistant, decomposing an engagement across a reasoning chain, a generation chain and a guidance chain so that each phase of an assessment is planned and executed incrementally [1]. Both contributions establish that language models can produce *plausible* security actions and can operate tooling in controlled environments, and both are representative of the generation-side focus of the field: output is produced for a human operator to review, and correctness is handled informally. Neither system subjects its output to deterministic schema, scope and authorisation validation before machine consumption, which is precisely the gap that a copilot feeding an *automated* toolchain must close. The IRE inverts this priority: it treats the model's generation as a candidate that must pass checkable predicates (Section 2.2) and only then becomes an actionable intent.

Attacks on language-model-integrated applications

The second body of work establishes that unvalidated model output is dangerous in integrated systems. Liu *et al.* formalised prompt-injection attacks against LLM-integrated

applications and systematically benchmarked a set of common defences, finding that prevention-based defences were weak and detection-based defences incomplete [5]. Greshake *et al.* demonstrated *indirect* prompt injection, in which an attacker plants instructions inside content—including web pages and documents—that an integration later retrieves and feeds to the model, and showed compromise of real-world assistant-style integrations [4]. Kang *et al.* showed that dual-use attacks that combine well-known computer-security techniques bypass in-the-wild content safeguards of deployed models with near-complete success [16]. The OWASP Top 10 for LLM applications consolidates these classes—prompt injection first among them—as first-class application risks rather than theoretical concerns [6].

The implication for this dissertation is direct. A security copilot that consumes an utterance, asks a model to classify it, and dispatches the result is exactly the kind of integration these attacks exploit: a single adversarial utterance can steer the classification output, and any downstream consumer will then act on the attacker-chosen intent unless an independent, non-model check intervenes. The relevant design response therefore places deterministic authorisation and validation *after* generation, so that even a perfectly diverted model cannot manufacture an accepted intent that violates syntax, scope or role policy (the safety invariant introduced in Chapter 4).

Detecting fabrication and measuring uncertainty

The third body of work supplies the theoretical toolkit for catching fabricated output. Ji *et al.* provide the taxonomy of hallucination in natural language generation that distinguishes intrinsic from extrinsic fabrication [3] (Section 2.2). Sampling-based detection uses the model's own output statistics: SelfCheckGPT scores the consistency of sampled responses to flag ungrounded claims [7]; semantic-uncertainty methods embed responses and cluster meaning-equivalent outputs to measure disagreement [8], and semantic entropy applies this principle to detect confabulations in free-form text [9]. Self-consistency decoding accumulates agreement across samples and rewards the majority answer [10].

What these methods share is an implicit budget: they assume either interactive generation with a user, or the freedom to produce several responses per query and cluster them. Each additional sample multiplies the inference cost of a CPU-only model on the operator's laptop, so the design of this dissertation must transpose the disagreement intuition into a *single* guarded classification: restricted sampling (a bounded number of responses), a

cheap embedding-based agreement gate, and a deterministic escalation path that does not depend on a large verifier model.

Grounding via retrieval and embeddings

A complementary line of work grounds model answers in external corpora rather than in self-consistency. Retrieval-augmented generation (RAG) conditions generation on retrieved passages so that answers can be traced to sources [11], and sentence embeddings provide the similarity signals such systems rely on [12]. RAG is attractive for factual Q&A, but it introduces retrieved content into the prompt at inference time—an additional prompt-injection surface [4]—and presupposes corpus infrastructure and embedding services that contradict the offline, single-device boundary adopted here (Section 1.7). The IRE therefore performs no retrieval at inference time and uses embeddings only for the agreement and cache signals.

Assessment

Taken together, the related research establishes three things: copilots can generate plausible security actions but do not validate them for machine consumption; integrated systems are exploitable through adversarial and indirect-injection utterances, and published defences are brittle; and fabrication can be detected in principle through sampling and semantic agreement, at a cost that grows with the number of samples. What the surveyed literature does not measure, to the best of the review conducted here, is the behaviour of a *guardrail stack*—a full deterministic validation pipeline with authorisation checks and a bounded self-consistency gate over a small offline model—under a strict single-inference latency budget, nor the isolated contribution of each guard layer, nor whether the achieved safety transfers between different offline model backends. Those measurements are precisely RQ1–RQ3, and Section 2.5 consolidates the gap they define into the conceptual framework used in the remainder of this dissertation.

[TODO: At the final AUDIT step, re-verify the characterisation of Liu et al. (defence benchmark) and Kang et al. (dual-use bypass) against the full text of their respective publications, since the descriptions here are relayed through the project's research paper rather than a direct reading of each source.]

2.4 Existing Systems, Products and Techniques

This section examines the systems, products and techniques that currently occupy the space in which the Intent Recognition Engine operates. They are reviewed against the same inclusion criteria as the wider survey (Section 2.1): relevance to at least one research question and a verifiable primary source reporting either an empirical evaluation or an authoritative specification. Where a surveyed option is a technique rather than a delivered product, the assessment asks whether it can be combined with the deployment constraint adopted in this dissertation—offline, CPU-only, single-inference, retrieval-free operation.

Deterministic security tooling

The techniques that the IRE ultimately drives are mature and deterministic. Nmap is the canonical network-discovery and scanning tool, whose techniques—host discovery, port enumeration, service and version detection—define the target values an intent contract must carry [13]. Metasploit is the canonical exploitation-framework driver, whose modules and options parameterise the most privileged class of intents [14]. Both expose structured command interfaces (a tool name, target, options and constraints) rather than free-form prose, and neither embeds a natural-language layer of its own. In current practice the mapping from operator intent onto these structured interfaces is performed by a human analyst or by a scripted wrapper, and any authorisation policy has to be wired externally; RFC 1918 [15] provides only the technical basis for distinguishing internal from external addressing. These tools therefore constitute the deterministic "ground truth" that a translation layer must produce—and the reason a translation error (Chapter 4) becomes a machine-dispatched error.

Language-model copilots for security

The recent systems that add natural language to this toolchain are generation-side copilots. PENTESTGPT decomposes a penetration-testing engagement across reasoning, generation and guidance chains and executes each phase incrementally, producing structured guidance that a human operator reviews [1]. The sparring-partner system demonstrated by Happe and Cito is a closed loop in which a model proposes low-level actions that are executed automatically against a deliberately vulnerable machine, up to root access [2]. Both establish that a model can drive security tooling in controlled environments, and both treat output correctness informally: the intended consumer of the output is human, and neither system subjects generation to deterministic schema,

scope or authorisation validation before an action is taken. As products, they are representative of the "AI assistant" category of security copilots rather than of an automated, machine-consumed intent layer.

Model-integration techniques: retrieval and tool calling

Beyond security copilots, two integration techniques are common to LLM-driven automation generally. Retrieval-augmented generation (RAG) conditions answers on external corpora so that claims can be traced to sources [11], using sentence embeddings as the similarity signal [12]; tool-calling scaffolds connect a model to external commands through function-call interfaces. As deployed techniques both widen the integration surface: retrieved content enters the prompt at inference time and is therefore an indirect-injection vector [4], and tool-calling binds a model action to a machine effect with no intermediate validation stage. The IRE is deliberately retrieval-free and does not render free-form model output to a downstream shell, which narrows these channels to the classification of a single utterance (Section 2.2).

Guardrail techniques for model-integrated applications

At the application layer, guardrail practice is codified by the OWASP Top 10 for LLM applications, which catalogues prompt injection (LLM01), insecure output handling and related classes as first-class risks rather than theoretical concerns [6]. The defence techniques surveyed in Section 2.3 are the prevention-detection families benchmarked by Liu *et al.* [5]—shown in that study to be individually insufficient—together with detection techniques for indirect injection [4] and the empirically demonstrated bypass of deployed content safeguards by dual-use attacks [16]. What these techniques have in common is that they act as app-layer checks over a model's behaviour after the fact. None of them is integrated as a deterministic validation stack that sits *between* model generation and machine dispatch and that makes syntax, scope and authorisation independently decidable (Section 2.2).

Fabrication-detection and uncertainty techniques

A separate product-facing technique family detects fabricated output. SelfCheckGPT scores the mutual consistency of sampled responses [7]; semantic-uncertainty and semantic-entropy methods measure answer disagreement over meaning-equivalence clusters [8, 9]; and self-consistency decoding accumulates agreement across samples [10]. These are deployed as techniques that assume repeated or interactive generation.

Two external references complement them for the value spaces relevant here: the National Vulnerability Database (NVD) is the authoritative registry of CVE identifiers [17], and the Exploit Prediction Scoring System (EPSS) scores the likelihood that a vulnerability will be exploited [18]. Both are the deterministic, offline-unfriendly references that a fabricated-identifier guard would consult if the offline boundary were relaxed (Chapter 6).

Comparative assessment

Table 2.1 consolidates the judgments from the subsections above against the properties that matter for the RQ1–RQ3 measurement (Section 1.6). "Schema check" and "scope/authorisation enforcement" record whether a system subjects machine-consumed output to structural and policy checks; "offline CPU" and "retrieval-free" record whether the system can run under the deployment constraint adopted here.

Table 2.1: Surveyed existing systems and techniques against the properties adopted in this dissertation. Judgements summarise the cited primary sources and are re-verified at the final AUDIT.

System / technique	Human review	Schema check	Scope / authz	Offline CPU retrieval-free
Nmap / Metasploit	–	Yes (interface)	External only	Yes
PENTESTGPT	Yes	No	No	–
Sparring partner	Partial	No	No	–
RAG agents	Partial	No	No	No
Fabrication detection	–	No	No	Partial
IRE (this work)	No	Yes	Yes	Yes

In summary, the surveyed landscape contains tools that are deterministic but require manual intent mapping, copilots that map intent but consume human review, integration techniques that connect models to tools without an intermediate validation stage, and defence techniques that check model behaviour after the fact. None combines a deterministic schema, scope and authorisation validation stack with offline, CPU-only, single-inference and retrieval-free operation over a closed intent taxonomy. That combination is the space the existing-systems survey leaves open and is consolidated in Section 2.5.

[TODO: If the assessment rubric requires a survey of *named* commercial security copilots (for example cloud SIEM/EDR copilot products), provide verifiable primary sources for each; none are currently in the reference database, so no commercial product is named here.]

[TODO: At the final AUDIT step, re-verify the comparative-table judgements in Table 2.1 against the full text of each cited source.]

2.5 Gap Synthesis and Conceptual Framework

The review in this chapter converges on three findings. First, the language-model copilots surveyed [1, 2] generate plausible security actions but do not validate them for machine consumption: their output is produced for a human reviewer, and correctness is handled informally (Section 2.3). Second, applications that integrate a model with tooling are exploitable through adversarial and indirect-injection utterances, the surveyed defences are individually insufficient [4, 5], and the existing-systems survey shows that current guardrail techniques act as post-hoc application-layer checks rather than as a deterministic validation stage between generation and dispatch (Section 2.4). Third, fabrication-detection and agreement methods [7, 8, 9, 10] are available in principle, but they assume sampling or interactive generation whose cost collides with the offline, CPU-only, single-inference budget adopted here (Section 2.2).

The gap that follows from these three findings is the combination that the existing-systems survey leaves open: no surveyed system delivers a dedicated, isolated natural-language understanding layer that converts free-form offensive-security instructions into validated, schema-constrained, authorised machine contracts while running offline, retrieval-free, on a single inference budget. This dissertation addresses exactly that combination by designing and *measuring* a guarded intent-recognition engine.

Conceptual framework

The design chapters build on three principles established in Section 2.2.

First, the language model is treated as a *stochastic proposition generator*: a single generated response is a sample weighted by the model's beliefs rather than a deterministic verdict (Section 2.2). Nothing the model emits is trusted until it satisfies checkable predicates; sampling-based agreement [10] provides a confidence signal, but the generation itself is always a candidate rather than a decision.

Second, responsibility is divided along the decidability boundary identified in Section 2.2: syntax, format and authorisation are decidable predicates (schema, host and CIDR syntax, port ranges, network and scope policy, role permission), while only meaning is delegated to the model. The guarded pipeline therefore couples a small instruction-tuned model with deterministic stages—schema validation, regex and network validation,

scope enforcement, role-based access control—whose purpose is to contain model error before it can propagate to a planner or executor (Chapter 4).

Third, the design enforces a *safety invariant*: an intent contract is accepted only when it satisfies the conjunction of the deterministic security predicates—schema-valid, network-authorized, in-scope, role-permitted and non-adversarial. The model may propose any candidate, but it cannot force acceptance of a candidate that violates any predicate in the conjunction. This converts the open problem of making a single language model trustworthy into the closed problem of classifying a single utterance under a known policy—the reduction that justifies acting on model-produced security instructions in an automated toolchain.

On top of these principles the framework adopts the bounded-sampling resolution of the cost tension from Section 2.2: a small, bounded number of responses, a cheap embedding-based agreement gate, and a deterministic escalation path (the hallucination validator) that does not depend on a large verifier model (Section 2.3).

From framework to measurement

The framework yields three measurable claims, each mapped to the corresponding research question (Section 1.6):

- **Pipeline value (RQ1).** The guarded pipeline raises the probability that model output satisfies the full conjunction of security predicates relative to raw generation; this is quantified by comparing intent accuracy, flaw-catch rate and false-positive rate with and without the guards.
- **Layer contribution (RQ2).** The contribution of each guard layer is quantified by ablating layers in turn against a constant baseline, isolating the largest safety contributor and the layers that trade safety for usability.
- **Generality (RQ3).** The identical guardrail stack is applied to two distinct offline backends to determine how much of the measured safety is backend-independent rather than an artefact of a single model.

The evaluation protocol that operationalizes these questions is developed in Chapter 3, the implementation and automated test suite in Chapter 5, and the measurements themselves in Chapter 6. Together with the preceding sections, this concludes the review, sets out the conceptual framework that guides the design of the Intent Recognition Engine in Chapter 4.

[TODO: At the final AUDIT step, cross-check the informal safety invariant stated here against the full formal definition to be developed in Chapter 4.]

3. RESEARCH METHODOLOGY

3.1 Research Design

This chapter defines how the research questions posed in Section 1.6 are answered through measurement. The dissertation is a design-and-evaluation study: it designs and implements an engineered artefact—the guarded intent-recognition engine introduced in Section 1.1—and evaluates that artefact empirically under controlled, reproducible conditions. The artefact is the object under study, and the controlled experiment over an annotated data set is the measurement instrument. The design is committed in advance: a fixed protocol applied to a fixed data set with a versioned evaluation harness (Section 3.7). The study is deliberately not a hypothesis-testing one; its outcomes are quantitative point estimates from a committed experiment record, and no statistical significance claim is attached to them (Sections 1.6 and 1.7).

System under study

The object evaluated is the guarded pipeline. At the level required to fix the measurement design, the pipeline is a seven-stage core that converts a free-form utterance into a validated intent—normalisation, alias resolution, model inference, parsing, schema validation, regex and entity validation, and scope enforcement—wrapped in five middleware components (the adversarial detector, pre-inference role check, session context, the semantic cache, and the post-validation role check) under a two-sided role-based access-control (RBAC) policy. The measurement design needs only the guard predicates: their presence or absence defines the treatment conditions. The complete stage-by-stage definition and interface contract are developed in Chapter 4.

Following the framework of Section 2.5, let $\mathcal{X}$ denote the utterance space, $\mathcal{Y}$ the closed intent taxonomy, C the set of classified flaw categories and ρ the caller role. The decoder is a stochastic map $f : \mathcal{X} \to \mathcal{Z}$; the deterministic guards are the predicates g_5, g_6, g_{6B}, g_7 over the schema, regex/entity, network and scope stages, the RBAC authorisation $b : \mathcal{R} \times \mathcal{Y} \to \{0, 1\}$, and m_1 the adversarial-input detector. The guarded decision function is

$$\Phi(x) = \begin{cases} y \in \mathcal{Y} \text{ accepted}, & \text{if } G(z, x) \wedge b(\rho, y) \wedge \neg\, m_1(x), \\ e \in C \text{ rejected}, & \text{otherwise}, \end{cases} \tag{3.1}$$

where $G(z, x) = g_5(z) \wedge g_6(z) \wedge g_{6B}(z, x) \wedge g_7(x)$. The safety invariant that the design enforces is that every accepted intent satisfies every security predicate:

$$\text{Accepted}\big(\Phi(x)\big) \;\Rightarrow\; \bigwedge\nolimits_{k \in K} P_k(x), \qquad K = \{\text{S5}, \text{S6}, \text{S6B}, \text{S7}, \text{RBAC}, \text{M1}\}. \quad (3.2)$$

Experiment 1 operationalises RQ1 as the gap in $\Pr[\bigwedge_{k \in K} P_k]$ between the raw decoder f and the guarded decision Φ; Experiment 3 operationalises RQ2 as the change in the catch rate attributable to each predicate when that predicate is removed from the conjunction (Section 3.3).

Treatment structure

The measurement design manipulates exactly three factors, and nothing else is allowed to vary between conditions:

- **Pipeline condition** (RQ1; Experiment 1). Raw single-shot model output is compared with the fully guarded pipeline, holding the backend, data set and thresholds fixed.
- **Backend** (RQ3; Experiments 2 and 3). Two offline, instruction-tuned backends—`qwen2.5-coder:7b` and `gemma4:e4b`—are compared under the identical guardrail stack and identical prompts.
- **Configuration** (RQ2; Experiment 3). The fully guarded baseline is compared with seven ablation conditions, each removing exactly one guard component: the scope guard, RBAC, the network validator, the hallucination validator, the deterministic validation stack as a combined block, the self-consistency gate, and the semantic cache. The set of ablation conditions is fixed in advance so that each layer's marginal contribution is measured against a constant baseline.

The dependent variables are the three rates that measure whether the system "does the right thing and does not do the wrong thing" under a guard: intent accuracy over the well-formed set, the flaw-catch rate over the flawed set, and the guard false-positive rate over the well-formed set. Their formal definitions and the strictness of the flaw-catch criterion are given in Section 3.4. A fourth experiment (Experiment 4) measures catch separately for each flaw category, so that the aggregate ablation result is interpreted per class rather than through a single number.

Control variables

To keep every comparison interpretable, the following are held constant across all conditions: the evaluation data set (Section 3.2); the hardware platform and the CPU-only Ollama runtime; the sampling temperatures (0.4 for the primary inference and 0.6 for the $k = 3$ consistency samples); the semantic agreement threshold; and the operator-configured engagement scope and RBAC policy under evaluation. Only the manipulated factor differs between conditions.

Measurement epistemology

Deterministic guard layers produce an exact outcome for a fixed input; the stochastic model component is executed once per input and condition. Every figure reported in this dissertation is therefore a single-run point estimate without seed sweeps or confidence intervals. The design compensates by committing the complete protocol, data set and harness to a versioned evaluation manifest, so that the record can be reproduced, audited and extended (Section 3.7). This choice is consistent with the evidential boundary declared in Section 1.7, and its consequences for how the results may be read are discussed in Chapter 7.

Chapter organisation

The remainder of this chapter defines the study materials and the constitution of the data set (Section 3.2), the experimental procedure and ablation protocol (Section 3.3), the metrics (Section 3.4), the method of analysis applied to the recorded measurements (Section 3.5), the ethics, privacy, security and responsible-AI considerations (Section 3.6), and the reproducibility and research-artefact commitments (Section 3.7). The implementation of the system under test is described in Chapter 5, the recorded outcomes in Chapter 6, and their interpretation against RQ1–RQ3 in Chapter 7.

[TODO: At the final AUDIT step, confirm the holdout definition used for Experiments 1 and 2 against the committed experiment record: the canonical 1500-record draft states that the stratified full 1500-prompt set was used, whereas the older 200-record paper draft reports a 200-prompt holdout. The chapters must quote whichever definition the versioned evaluation manifest records.]

3.2 Data and Study Materials

This study involves no human participants. The study materials are the annotated evaluation data set that drives the recorded experiments, the deployment platform on which they run, and the two language-model backends under test. The data set is author-authored rather than sampled from operational traffic, an assumption declared in Section 1.7; it is versioned with the project source and regenerable from the synthetic-generation script used to author it (Section 3.7).

Constitution of the data set

The evaluation data set D has two disjoint subsets. The well-formed subset D^+ contains 600 free-form security requests, distributed across the nine top-level intents of the closed taxonomy used by the system under test (scanning and discovery, exploitation, password attack, directory bruteforce, CVE lookup, reconnaissance, phishing, network defence, and miscellaneous). Each record in D^+ is annotated with a gold intent label $y^*(x)$.

The flawed subset D^- contains 900 deliberately malfunctioning requests, distributed across the eight flaw categories in Table 3.1. Each record in D^- carries two gold labels: an intent label $y^*(x)$ and a flaw label $c^*(x)$ drawn from the eight-category flaw taxonomy C, which is the rejection category a correct pipeline is expected to assign (Section 3.4). The eight categories span the failure modes that matter in an automated security toolchain: adversarial phraseology intended to divert the model; requests whose target or technique falls outside the operator-declared engagement scope; requests too ambiguous or under-specified to resolve; internally contradictory requests; fabricated parameter values; fabricated CVE identifiers; requests whose phrasing mismatches the structure of the stated intent; and requests an unauthorised role attempts to issue. The mapping between each category and the guard layer or layers responsible for catching it is a hypothesis tested by the per-class experiment (Section 3.3) and reported in Chapter 6.

The flawed categories in the 1500-record set are near-evenly sized (113 or 112 records each, the D^- classes summing to 900). Per-class catch rates are therefore reported against each category's own denominator rather than as a single aggregate, and cross-category comparisons are only meaningful against those denominators (Section 3.4). This is also why the per-class experiment supplements the aggregate ablation: no flawed category is large enough on its own to dominate the aggregate figure.

Table 3.1: Experimental data set composition ($N = 1500$). D^+ holds 600 well-formed requests across the nine-intent taxonomy; D^- holds 900 deliberately flawed requests across eight flaw categories.

Category	**Records**
D^+ well-formed requests	600
Adversarial injection	113
Out-of-scope target/technique	113
Ambiguous / under-specified	113
Contradictory request	113
Fabricated parameter	112
Fabricated CVE	112
Type mismatch (string vs. intent)	112
RBAC violation	112
Total	1500

Annotation practice

Gold labels are assigned at authoring time under a fixed category definition, so that every record in a category is flawed for the stated reason and no record is flawed for a reason that belongs to another category. Because the flaw-catch metric requires the assigned rejection category to match the gold label exactly (Section 3.4), borderline phrasings that sit between two categories contribute label noise, most plausibly at the boundary between the adjacent semantic classes (ambiguous and contradictory); the residual weakness of those classes is reported in Chapter 6 and their interpretation is qualified accordingly.

Platform and backends

All recorded measurements were obtained on a single consumer laptop—an ASUS VivoBook X513EA (Intel Core i5-1135G7, four cores and eight threads, 24 GB DDR4, Windows 11)—running Ollama in CPU-only mode; no GPU was used. The two backends under test are `qwen2.5-coder:7b` (4.7 GB on disk) and `gemma4:e4b` (9.6 GB on disk), the build to which the `gemma4:latest` alias resolved at evaluation time. The Ollama version and the per-model image digests as fetched are pinned in the evaluation manifest, which is the authoritative anchor for reproducing the study (Section 3.7). The engagement scope, the RBAC policy and the semantic threshold used by the self-consistency gate are configuration materials of the same manifest; they are held constant across all conditions (Section 3.1).

[TODO: At the final AUDIT step, reconcile the composition reported here (N = 1500; D^+=600; D^-=900 with the eight flawed categories) against the committed synthetic dataset generation script and its generated artefacts, and confirm the annotation conventions against the golden dataset stored with the project source.]

3.3 Experimental Procedure

This section defines how the materials of Section 3.2 are processed into the recorded measurements reported in Chapter 6. The procedure is fixed in advance and versioned: every pass is executed by the evaluation harness against the same data set in the same record order, and the decision recorded for every record is written to the committed experiment record (Section 3.7).

Execution model

For each record, the harness issues the request to the service once per condition, and buffers the single recorded decision. In the *guarded* condition the pipeline runs end-to-end: the pre-inference role check, the adversarial detector, the seven core stages including the self-consistency gate, and the post-validation role check. The outcome is either an accepted intent $y \in \mathcal{Y}$ (only when the conjunction of guards holds, Equation 3.1) or a rejection $e \in C$ carrying the guard-assigned flaw category. In the *raw* condition, used by Experiment 1 only, the harness records the model's single-shot structured output and applies no guard predicates: no rejection category exists in raw output, so no raw output can be classified as flawed.

Experiment set

- **Experiment 1 (pipeline value; RQ1).** For each backend, the raw condition is compared with the guarded condition over the full stratified 1500-prompt set of Section 3.2 (600 well-formed and 900 flawed records). This quantifies the contribution of the guardrail stack as the gap in the probability that output satisfies the full conjunction of security predicates (Section 3.1).
- **Experiment 2 (backend head-to-head; RQ3).** The guarded stack is held constant while the two backends are compared on the identical full holdout with identical prompts. This isolates the effect of the underlying model from every other pipeline component.
- **Experiment 3 (layer ablation; RQ2 and RQ3).** Both backends are evaluated on the full 1500-record set under eight configurations: the fully guarded baseline and

seven ablation conditions, each removing exactly one guard component—the scope guard, RBAC, the network validator, the hallucination validator, the combined deterministic validation stack (schema, regex/entity, network and scope validation disabled together as one block), the self-consistency gate, and the semantic cache. The combined validation-stack condition is a *joint* ablation that bounds the stack's aggregate contribution; it is not a single layer. The marginal contribution of a layer is

$$\Delta\text{Catch}_\ell = \text{Catch}_{\Pi\setminus\ell} - \text{Catch}_\Pi, \tag{3.3}$$

so that a negative value indicates the layer contributes to the catch rate.

- **Experiment 4 (per-class catch; RQ2 evidence).** Catch is measured separately for each of the eight flaw categories on a dedicated deep-dive subset of flawed prompts, so that the aggregate ablation result is interpreted against each category's own denominator rather than a single figure.

Self-consistency gate

The self-consistency gate adapts the sampling-based consistency strategy of Wang *et al.* from generative reasoning to structured classification: for each primary inference (temperature 0.4), $k = 3$ additional responses are sampled at temperature 0.6, embedded, and compared for agreement. The recorded agreement signal is

$$\alpha(x) = \binom{k}{2}^{-1} \sum_{1 \le i < j \le k} \mathbf{1}\big[d_{ij} \le \theta_s\big], \tag{3.4}$$

where $d_{ij} = 1 - \cos\big(\mathbf{e}(z_i), \mathbf{e}(z_j)\big)$, $\mathbf{e}(\cdot)$ is the sentence embedding, and θ_s is the semantic threshold pinned in the evaluation manifest; a raw single sample has $\alpha = 1$ by construction. Agreement below the threshold demotes confidence and routes the request to the hallucination validator, which assigns the output to one of six error classes (impossible CVE, impossible parameter range, unverifiable value, contradictory fields, out-of-scope reference, unresolved type) and selects the corresponding response—BLOCK, WARN, or ESCALATE (the policy is a configuration of the manifest and is exercised by the automated test suite).

Adversarial detection

The adversarial detector precedes the model where possible. Its lexical layer assigns suspicion weights of 0.8–1.0 to prompt tokens, its structural layer scores sentence topology and imperative-but-unclassifiable phrasing at 0.3–0.9, and the scoring layer

fuses the two signals cumulatively. A cumulative score above 0.7 flags the request for heightened scrutiny, and a score of at least 0.95 auto-blocks it without paying for model inference.

Fixed operational parameters

The following are held constant across every condition (Section 3.1) and pinned in the evaluation manifest: the primary sampling temperature (0.4); the consistency sampling count ($k = 3$) and temperature (0.6); the semantic agreement threshold θ_s; the adversarial-detector flag and auto-block thresholds (0.7 and 0.95); the operator-configured engagement scope; the RBAC policy; and a per-stage timeout so that an unresponsive model cannot stall the pipeline. Design decisions that would add a further model call were rejected on latency grounds, which bounds the sampling budget at the fixed k above.

Recording and reproduction

Each configuration is executed once per backend, with the stochastic model component sampled once per record and condition (Section 3.1). Per-record decisions are appended to the experiment record in manifest-defined order, and every table reported in Chapter 6 is regenerated from that record by a single command (Section 3.7), so the measured statements of this dissertation are, in principle, reproducible without re-running the model.

[TODO: At the final AUDIT step, (i) confirm the composition of the Experiment 4 deep-dive flawed subset and its per-class denominators against the committed manifest, since the canonical 1500-record draft does not enumerate them, and (ii) re-verify how the raw-condition false-positive figures were computed by the evaluation script given that the metrics definition reserves false positives for guard flags (Section 3.4).]

3.4 Measures and Evaluation Metrics

Three rates constitute the dependent variables of the study (Section 3.1). Let $y^*(x)$ and $c^*(x)$ denote the gold intent and flaw labels of record x, and $\hat{y}(x)$ and $\hat{c}(x)$ the intent and flaw category assigned by the system. In the guarded condition an accepted record carries a recognised intent $\hat{y}(x)$ and no flaw category; a rejected record carries a classified flaw category $\hat{c}(x)$ and no intent. Two conventions follow: a well-formed

record that a guard flags is counted as not recognised (so it never inflates accuracy), and a flawed record that is rejected under *any* other category is counted as not caught. All rates are reported as percentages.

Intent accuracy

The intent accuracy D_{Acc} is computed over the legitimate-input set D^+:

$$D_{\text{Acc}} = \frac{100}{|D^+|} \sum_{x \in D^+} \mathbf{1}[\hat{y}(x) = y^*(x)] \,, \tag{3.5}$$

where $\mathbf{1}[\cdot]$ is the indicator function. It measures how often the pipeline both accepts a well-formed request *and* recovers its intended class—the system "doing the right thing."

Flaw-catch rate

The flaw-catch rate is computed over the flawed-input set D^-:

$$\text{Catch} = \frac{100}{|D^-|} \sum_{x \in D^-} \mathbf{1}[\hat{c}(x) = c^*(x)] \,. \tag{3.6}$$

The strictness of this criterion matters for interpretation: a request that is rejected, but under the wrong category, is a miss. The metric therefore rewards not merely refusal but *correctly diagnosed* refusal, which is the property an automated toolchain needs—a downstream consumer must know *why* a request was refused, not only that it was. Aggregate catch is complemented by per-class reporting (Experiment 4) because the flaw categories of the data set carry their own denominators and no single figure expresses the coverage of every category (Section 3.2).

False-positive rate

The false-positive rate is computed over the legitimate-input set D^+:

$$\text{FP} = \frac{100}{|D^+|} \sum_{x \in D^+} \mathbf{1}[\text{a guard flags } x] \,, \tag{3.7}$$

measuring how often a guard erroneously flags a well-formed request (a "shadow-ban"). A flagged well-formed request is a conservative over-block, not a misclassification toward a privileged intent: the guard either marks the request for heightened scrutiny or rejects it outright; it never re-approves it as a different intent. FP is therefore the

usability counterpart of the two safety rates, and the design goal is simultaneously high D_{Acc} and high Catch with low FP.

Ablation measure

The per-layer contribution to catch is the delta defined in Section 3.3, $\Delta\text{Catch}_\ell = \text{Catch}_{\Pi\setminus\ell} - \text{Catch}_\Pi$ (Equation 3.3). Because the ablation removes exactly one guard predicate from the conjunction of Equation 3.2, the delta isolates the marginal effect of that predicate on the strict category-match criterion. The paper drafts report an approximate run-to-run noise band of ±1–2 percentage points for the ablation configuration, which is carried into the results chapter as an interpretation aid rather than a formal confidence statement (Section 3.5).

Operational measures

In addition to the accuracy metrics, the harness records operational cost for each cache-miss request: end-to-end latency from request receipt to the returned decision, aggregated as percentiles over the run. These figures are recorded per backend and configuration in the evaluation manifest. Because the author has not yet finalised the latency figures for the canonical 1500-record evaluation, no latency percentile is quoted in this dissertation at present; the values will be reported in Chapter 6 once they are confirmed against the manifest. Semantic-cache-hit latency is recorded separately, since a cache hit bypasses model inference entirely.

[TODO: Latency figures for the canonical evaluation are PENDING from the author (AUDIT checkpoint: quote them in this section and in Chapter 6 only after the author confirms the values against the evaluation manifest).]

3.5 Data Analysis

The analysis applied to the recorded measurements is descriptive and quantitative: every claim in the results chapter is a comparison of rates computed from the committed experiment record. In line with the design decision of Section 3.1, no inferential statistics are applied—no significance tests, no confidence intervals and no seed sweeps are reported. Each experiment therefore yields point estimates, and the analysis states, for each comparison, the rule by which a difference is read.

Pairwise comparisons

Experiment 1 (raw against guarded) and Experiment 2 (Gemma against Qwen under an identical stack) are read as side-by-side rates per backend and per metric. Because the pipeline's stochastic component is sampled once per record and condition, the reported differences are point comparisons; the analysis does not attach uncertainty bounds to them. The results are presented as point deltas (percentage points) and interpreted qualitatively where the underlying mechanism is deterministic: for a guard whose outcome is a pure function of its input, a difference on the affected subset is read as exact behaviour, not as a sampling artefact.

Ablation deltas

For Experiment 3, the analysis uses the per-layer delta of Section 3.3, ΔCatch_ℓ (Equation 3.3), together with the accompanying movement in D_{Acc} and FP. The recorded run-to-run noise band of approximately ±1–2 percentage points is the interpretation boundary: a delta outside the band is read as a layer effect, while a delta inside the band is not distinguished from noise. This is a pragmatic reading rule, not a statistical one, and is carried into the results chapter as such. Two deltas require special handling. First, the combined validation-stack condition disables S5–S7 as one block, so its delta is attributed to the block as a whole, never to a single stage. Second, a *positive* delta (catch *rises* when a layer is removed) signals an interaction rather than a contribution; the analysis records where this pattern appears—the paper runs observe it for the scope guard, the network validator, the self-consistency gate and the semantic cache in both backends, with FP rising alongside it—and treats those layers as false-positive suppressors until the underlying mechanism is confirmed. The candidate explanation (a pre-emptive generic rejection prevents the later stage from assigning the record's true specific label) is recorded as a hypothesis, not an established result.

Disaggregation

Two disaggregations are fixed in advance because the canonical drafts report them. Catch is analysed per flaw category against each category's own denominator (Experiment 4), never as an aggregate number alone, because the categories carry different denominators and the weak semantic classes would be hidden in a single figure. The D_{Acc} layer ordering is analysed *per backend* rather than as one ranking: the canonical drafts find no single layer that dominates accuracy across both backends, so a combined ranking

would mislead. Consistency claims across backends are restricted to the ordering of catch-rate effects, and any claim about accuracy ordering is stated backend-specifically.

Result-to-question mapping

The analysis produces the evidence for each research question as follows: RQ1 is answered by the raw-versus-guarded deltas of Experiment 1; RQ2 by the ablation deltas, the per-class disaggregation and the recorded interactions of Experiment 3–4; and RQ3 by comparing whether the ordering of effects carries across the two backends (Experiment 2 and the per-backend ablation columns of Experiment 3). The tables that instantiate this analysis are generated directly from the committed record by the reproduction command (Section 3.7); the interpretation of validity threats that the analysis cannot address is deferred to Chapter 7.

3.6 Ethics, Privacy, Security and Responsible AI

Data ethics

The study involves no human participants and processes no personal data. The evaluation data set (Section 3.2) is author-authored and synthetic: every record is a constructed security request, not a sample of operational traffic, and no record captures information about an identifiable individual. No consent procedures, participant recruitment or data-protection clearance for personal data were therefore required. The synthetic set is versioned with the project source so that the same records, and only those records, underpin every measurement reported in this dissertation (Section 3.7).

Dual use

The subject of the dissertation is a natural-language gateway to security tooling, including scanning, reconnaissance and the drivers of exploitation frameworks within an operator-declared scope. The capability is dual use by nature. Three design and practice decisions bound this risk. First, the system is deployment-gated: every action is subject to role-based access control and engagement-scope enforcement, and the safety invariant (Equation 3.2) ensures that an accepted intent is authorised and in scope before anything downstream may act on it. Second, the evaluation is conducted entirely offline against synthetic prompts in a lab harness; no live, third-party or production system was ever contacted, and the design assumption that downstream components honour the rejected-intent contract is declared in Section 1.7. Third, the recorded

failure modes—including the residual weakness of the semantic flaw classes reported in Chapter 6—are published as part of this dissertation rather than suppressed, in keeping with established practice in dual-use security research.

Privacy

The pipeline is deliberately offline and single-device: no utterance or result leaves the device, nothing is retrieved from an external service at inference time, and the operator's requests are not transmitted anywhere by the system (Section 3.1). The only persistent assets are the model weights, the operator-configured policies and the session context. No personal data is collected, stored, shared or required for any experiment reported here.

Security in the research process

The study is conducted under a defined threat model (Sections 2.5 and 3.1): the attacker controls the single user utterance, not the device, prompts, policies or model weights. The evaluation harness exercises the pipeline as a lab component, with no free-form model output ever rendered to a downstream shell and a per-stage timeout guarding against a stalled model. No secrets beyond the model weights are held, and the research artefacts (code, data set, evaluation scripts and manifest) are versioned so the experiment record can be audited independently.

Responsible use of artificial intelligence

The guarded design is itself a responsible-AI measure: the model's output is treated as an untrusted proposition until validated deterministically, which is the stance this dissertation argues every LLM-integrated security automation should adopt (Sections 2.5 and 4). Responsible reporting is observed throughout: measurements are presented as single-run point estimates without overstated confidence (Sections 3.1 and 3.5), limitations and residual failure modes are declared rather than hidden (Section 1.7), and the safety/usability trade-off is reported honestly, including the deliberate preference for conservative over-blocking over risky under-blocking (Section 3.4).

In preparing this dissertation, AI-based tools were used for language correction, drafting support and the development of the artefacts described here. The use follows the institution's policy on AI tools and is disclosed in the applicable declaration (Appendix B).

Compliance

Because the study involves no human participants, no personal data and no intervention in live systems, it falls outside the class of work requiring institutional research-ethics approval, and the applicable integrity declaration is included in Appendix B.

[TODO: At the final AUDIT step, confirm with the faculty (i) that no institutional research-ethics approval is required for a purely synthetic, no-participant study of this kind, and (ii) the exact wording required for the AI-tools disclosure in Appendix B under the current institutional AI policy, and align the disclosure in this section with it.]

3.7 Reproducibility and Research Artefacts

The study is reproducible by design. Every material that produced the measurements reported in this dissertation is versioned with the project source, and a single command re-executes the evaluation and regenerates every table used in the results chapter. The purpose of this section is to state exactly what the artefacts are, what the evaluation manifest pins, and what the boundaries of reproducibility are given the stochastic component of the system.

Artefact set

The following artefacts constitute the research record:

- **Source code.** The Modular Python service (FastAPI) with the separation between core stages, guard middleware and the language-model adapter (Chapter 5).
- **Data set.** The N=1500 annotated evaluation data set (Section 3.2) together with the synthetic-generation script used to author it.
- **Evaluation harness.** The scripts that execute every experiment, apply the procedure of Section 3.3 and compute the metrics of Section 3.4.
- **Evaluation manifest.** The single configuration document that pins every experimental constant (below).
- **Committed experiment record.** The per-record decisions written by the harness, from which every reported table is regenerated without re-running the model.
- **Automated test suite.** The 482 automated tests (434 unit and 48 asynchronous integration tests, measured 2026-08-26) that exercise the guard paths, the RBAC matrix, the cache paths and the hallucination policy, and that gate the harness before any measurement is accepted.

Evaluation manifest

The manifest is the authoritative anchor for reproducing the study. It pins the Ollama runtime version and the per-model image digests as fetched, the hardware platform and operating system, the data set version, the sampling temperatures (0.4 primary and 0.6 for the k=3 consistency samples), the semantic agreement threshold θ_s, the adversarial-detector flag and auto-block thresholds, the operator-configured engagement scope and RBAC policy, and the per-stage timeout. No experimental constant is decided at run time (Section 3.1).

Reproduction procedure

A clean checkout of the versioned artefacts is sufficient: a single reproduction command runs the harness against the data set, recomputes every metric and regenerates every table that appears in Chapter 6. Because the per-record decisions are committed, the reported figures can also be regenerated directly from the experiment record without re-invoking the model, which makes the numerical claims of this dissertation auditable independently of the stochastic generator. The evaluation instruments used in that record are collated in Appendix A.

Boundaries of reproducibility

Two boundaries are stated plainly. First, the model component is stochastic: an independent re-run may yield point estimates that differ slightly from the single-run values committed here, which is precisely why the dissertation reports point estimates without confidence claims (Sections 3.1 and 3.5); the committed record guarantees that the values reported are real, measured values. The deterministic guard layers, by contrast, are exact functions of their inputs and reproduce bit-for-bit on the same input set. Second, absolute latency figures are bound to the evaluation hardware and runtime; the manifest pins the exact versions so that the record can be reproduced on comparable equipment, and cross-device comparisons require re-measurement rather than extrapolation (Section 3.4).

Integrity

The research artefacts contain no personal or sensitive data, and no record in the data set corresponds to an identifiable individual. The versioned artefact set, the committed experiment record and the reproduction command together constitute an auditable trail from the raw measurements to every number quoted in this dissertation; discrepancies

between the record and the text are defects to be corrected, not adjustments to be made. With the protocol fixed in this chapter, the following chapters develop the design of the system under test (Chapter 4), its implementation (Chapter 5), and the measurements and their interpretation (Chapters 6 and 7).

[TODO: At the final AUDIT step, verify on a clean checkout that the single reproduction command executes end-to-end, regenerates every table cited in Chapter 6, and that the committed experiment record is complete for every configuration reported.]

4. PRODUCT AND SOLUTION DESIGN

4.1 Requirements

Product context

NeuroShell IRE (the Intent Recognition Engine) is Component 1 of the NeuroShell Core Framework, a language-driven security-automation stack. The product is a domain-specific natural-language understanding gateway: it converts a free-text offensive-security command into a validated, schema-constrained intent contract that downstream components can consume without ever seeing free-form model output (Section 4.1). The contract-first boundary is the central requirement of the product: every output exposed to the toolchain must be machine-validated, and no unvalidated LLM text may reach it (Chapters 1 and 3).

The IRE runs as a local daemon on the operator's device, serving other components and human operators over HTTP. The complete NeuroShell framework places two further safety gates downstream of the IRE—a planner that enforces its own RFC 1918 and dangerous-pattern checks, and an executor that runs recovery-driven command execution—so the IRE is designed as the first, but not the only, enforcement point.

User classes

The product serves four human role classes in a strict privilege hierarchy (viewer, analyst, operator and admin) and two classes of machine consumer (downstream services and human-in-the-loop reviewers). Table 4.1 summarises the roles; the full intent-permission matrix and behavioural rules are given in Section 4.4.

Table 4.1: Role model for the IRE. The default role is analyst; unknown roles coerce to viewer (least privilege).

Role	Hierarchy	Privilege character
viewer	0 (least)	Observation only; pre-inference fast-fail for any intent
analyst	1 (default)	Passive recon, network scan and vulnerability audit
operator	2	All intents, including the three sensitive classes
admin	3 (most)	All intents plus model, policy and feature management

Functional requirements

The functional requirements are grouped under the product functions verified in the implementation (Section 4.2):

- **Core understanding.** Normalise the input utterance; resolve offensive-security aliases (for example `EternalBlue` to CVE-2017-0144) to canonical identifiers; classify the request into one of nine top-level intents; and refine it into a hierarchical sub-intent that drives downstream tool selection.
- **Safety and security.** Detect adversarial input before inference; enforce role-based access control both before and after inference; validate the model's output against typed schema, regular-expression, network and engagement-scope rules; classify every failed validation into one of six hallucination classes; and emit an audit record for every parse.
- **Intelligence and quality.** Maintain per-session context across turns; serve identical intents from a semantic cache; gate low-confidence decisions through a self-consistency stage; and expose an optional XAI attribution path.
- **System and integration.** Expose a REST API for parsing, execution and administration; push validated contracts to downstream components on a feature-flagged basis; persist an append-only audit trail; and make every guard individually disable-able through configuration.

Non-functional requirements

- **Performance.** Adversarial detection (M1) must complete in low milliseconds; the pre-inference RBAC fast-fail runs before any model cost; and cache hits must short-circuit the pipeline (Section 3.4).
- **Reliability.** A failure in non-critical middleware (session context, cache, adversarial detection) must degrade gracefully, not abort the request.
- **Security.** The service holds no credentials beyond an operator API key; rate limiting protects the model backend; and no free-form model output is ever returned to a caller (Section 1.7).
- **Operability.** Every stage and middleware is individually configurable via feature flags; the model family and engagement scope are administratively reconfigurable at runtime; and the whole evaluation is reproducible from a single manifest (Section 3.7).

4.2 Component Architecture within the Complete Solution

Position in the NeuroShell framework

The IRE is Component 1 of a four-component stack. Component 2 (the Dynamic Planner) consumes `PlannerContract` objects to synthesise an execution plan; Component 3 (AEERE) executes the plan with adaptive error recovery; and Component 4 (AVAE) performs post-execution vulnerability analysis. Downstream integration is optional and feature-flagged (`planner_enabled`, `component3_enabled`, `component4_enabled`), so the IRE is a complete product on its own and a drop-in first stage of the framework. Machine consumers are authenticated by a shared API key and receive only validated schemas; they never receive free-form LLM output.

Top-level architecture

Table 4.2 maps the verified pipeline stages to their implementing modules under `src/`. A deterministic core of seven serialised stages (S1–S7, where S6B is the network sub-validator of S6 rather than an eighth stage) transforms the raw utterance into a validated intent, while five middleware modules enforce mandatory safety and quality properties around the core. The guarded decision function and safety invariant (Equations 3.1 and 3.2, Chapter 3) are realised by exactly these components.

Processing order

Middleware runs around the core: M1 classifies the raw input, M2 fast-fails restricted roles before any model cost, M3 injects session context, M4 serves cache hits that short-circuit the LLM entirely, and the core (S1–S7, S6B) then runs in strict sequence. Following the core, M5 re-validates the detected intent against the role matrix, and the ContractBuilder produces the downstream contract. The appendix and the threat model (Sections 2.5 and 3.1) treat the downstream boundary: the IRE renders no free-form content and retrieves nothing, closing the indirect-injection and tool-injection channels.

4.3 Design Alternatives and Decisions

Local, offline decoding versus a hosted model

The first architectural decision is the decoder. A cloud-hosted model offers higher raw accuracy and easier scaling, but contradicts the product requirement that no request

leaves the device and no credentials be held (Sections 4.1 and 3.6). The chosen design runs a small model locally through Ollama in CPU-only mode, which reflects the intended field deployment and keeps the trust boundary on a single device. Two backends are kept configurable so generality can be measured (Chapter 3).

Structured contract versus free-form prose

Raw LLM output is plausible prose and a machine-dispatched action must not be built from prose. The chosen design forces the model to emit a fixed JSON schema (S3) and then validates it through S4–S7 before anything is returned. Only an output that survives these stages is exposed; otherwise a stage-specific classified error is returned. This converts a plausibility claim into a verifiable one and is the decision behind Equations 3.1 and 3.2.

Deterministic rules versus model self-checking

Two validation philosophies are available: ask the model to review its own output, or recompute the checks with deterministic rules. Self-checking is drift-prone and reuses the same faulty estimator. The chosen design recomputes every syntactic, network and scope check with deterministic code (S5–S7, S6B) that depends on no further LLM calls. This is why the deterministic guard path is bit-exact and auditable (Section 3.7).

No retrieval versus retrieval-augmented generation

Retrieval-augmented generation is a popular way to ground answers, but retrieved text is itself a prompt-injection vector [4], and a retrieval channel expands the offline trust boundary. The chosen design performs no retrieval at inference time; offline CVE grounding is loaded at startup only. This decision is what makes the threat model closed (Section 4.4).

Two-phase RBAC versus a single check

Role enforcement can be a single post-hoc check or a repeated invariant. The chosen design enforces RBAC twice: a pre-inference fast-fail (M2) that rejects restricted roles before any model cost, and a post-inference check (M5) that validates the final intent against the role matrix. A single bypassed guard is therefore not sufficient to manufacture an authorised action.

Self-consistency versus single-pass confidence

The model supplies a confidence score, but calibrated scores are unreliable for small offline models. The chosen design gates low-confidence decisions through a self-consistency stage that samples further responses and measures agreement, accepting the bounded latency cost only when triggered. This is the sampling-based disagreement intuition applied to a single structured classification rather than to free-form text [10].

Semantic caching

Identical intents recur in operator workflows. The chosen design adds a two-channel semantic cache (exact hash and embedding similarity) that short-circuits the LLM on hits, cut latency from seconds to tens of milliseconds per hit, and improves determinism; the cache is invalidated when the offline CVE-grounding database is reindexed. The latency figures are not quoted here pending author confirmation (Section 3.4).

4.4 Detailed Design

Core stages (S1–S7, S6B)

S1 normalises casing, whitespace and Unicode (NFKC) and standardises jargon terms into canonical phrases. S2 resolves aliases such as `nmap -sS` to a SCAN action and exploit surnames to canonical CVE identifiers via `data/alias_map.json`. S3 instructs the model to emit a fixed JSON schema, at the sampling configuration pinned in the evaluation manifest (Section 3.7). S4 recovers the JSON payload, stripping code fences, markdown artefacts and thinking tags. S5 enforces the typed `IntentSchema` contract: exactly one of nine `IntentType` values, a matched target type, ports in 1–65535, CVE syntax `CVE-YYYY-NNNN`, confidence in [0,1], and the invariant that `AMBIGUOUS` requires confidence below 0.5 while `REJECTED` requires a `rejection_reason`. S6 deterministically re-verifies every entity the model produced (IPv4/IPv6, CIDR, CVE, port, URL) and the absence of shell metacharacters. S6B applies engagement-scope and topology rules: targets must fall inside the configured engagement CIDR (default 192.168.0.0/16), the widest permitted scan prefix is 30 and the narrowest 32, and type/value mismatches fail here, generating `ARCH_WARNING` and `SCOPE_WARNING` entries. S7 is the final enforcement gate: it rejects out-of-scope targets in strict mode, warns in research mode, applies RFC 1918 membership [15], and re-scans the raw input as an injection gate.

Middleware

M1 is a three-layer adversarial detector operating before inference: a lexical layer scores known injection phrases and control tokens (weights 0.8–1.0), a structural layer scores encoded payloads, script-like constructs and imperative-but-unclassifiable phrasing (0.3–0.9), and a scoring layer fuses the signals; a cumulative score at or above 0.95 auto-blocks immediately, and 0.7 flags the request for downstream treatment (Section 3.3). M2/M5 implement RBAC in two phases around the core as described in Section 4.3; the policy lives in `config/rbac_policy.py` and rules that unknown roles coerce to `viewer`, the least-privileged class. M3 maintains per-session turn history with a TTL, injects the recent memory window into the prompt and supports anaphoric follow-ups such as "the same target". M4 serves exact-hash and cosine-similarity cache hits, evicts LRU-style, and is invalidated with the CVE-grounding hash. The sub-intent classifier refines the top-level intent into a hierarchical sub-intent (for example `NETWORK_SCAN.SYN_STEALTH`), which the ContractBuilder maps to a suggested tool and flags (Section 4.4).

Hallucination taxonomy and enforcement

Every failed validation is classified into one of six hallucination classes via the enforcement-policy configuration module (`config/enforcement_policy.py`) and becomes an audit-grade `ValidationFinding`. The policy maps classes to BLOCK/WARN/ESCALATE outcomes: fabricated parameters, contradictory action-target pairings, target-type mismatches and malformed fabricated CVEs hard-block; ungrounded confidence (syntactically valid but unindexed CVEs) warns; and out-of-scope targets warn in research mode but block in strict production mode. Escalations route through the admin escalation queue for explicit human approval.

Contract generation and administration

The ContractBuilder synthesises a downstream-ready `PlannerContract` with a strict, fully validated field set (intent, sub-intent, validated target, ports, modifiers, CVE IDs, tool hint, schedule, confidence, rejection reason and session ID), a human-readable description, a priority level (scheduled, critical for exploitation, high for CVE audits, medium otherwise) and a generated suggested command. Exploitation and password-attack flows are marked as requiring explicit human approval. Management is exposed through the REST API: parsing, execution, metrics, sessions, and admin views for RBAC, feature flags, models, enforcement policy, escalations, audit summary and the

engagement scope. An append-only audit logger persists a SHA-256 hashed form of operator content with the intent, role, latency and outcome as structured fields.

Deployment properties

The service runs headless, CPU-only, retrieving nothing at inference time and holding no credentials beyond the operator API key; its only persistent assets are the model weights, the authorization policy, the session context and the audit database. Because nothing is retrieved and no free-form output is rendered downstream, the two principal remote attack classes (indirect prompt injection via retrieved data [4] and tool injection via unfiltered text) are structurally reduced to classifying a single user utterance.

Boundaries of the design record

The audit rule for this chapter is that components are described only when their implementing module is present under `src/`. Four capabilities that appear in the project documentation are therefore flagged rather than described as implemented: `spaCy` text pipelining is referenced in the SRS dependencies but has no module in `src/`; the `jsonschema` runtime is a documented dependency while the actual validator in `src/` uses Pydantic models; LoRA fine-tuning exists as a configuration file (`config/lora_config.yaml`) but is planned Phase 2 work, not an implemented execution path; Mahalanobis anomaly detection and XAI token attribution appear as configuration (feature-flag `xai: false`) and tuning parameters, not as wired components. Where these names recur in discussion, they are treated as unimplemented design candidates.

4.5 Standards and Professional Practice

The design follows established standards where they apply. The SRS was written against the IEEE 830 recommended practice for software requirements specifications, and the architecture's private-address enforcement is grounded in RFC 1918 [15]. The threat catalogue that motivates the guard stack follows the OWASP Top 10 for LLM applications [6], and offline CVE grounding draws on the National Vulnerability Database as a canonical identifier source [17]. Security tooling references, such as the suggested scanning tools and the exploitation framework, follow the documented syntax of the respective vendors [13], [14] so that generated contracts are executable by the downstream components as intended.

Professional practice in this domain is primarily the ethics of dual use (Section 3.6): the system can drive offensive tooling, so the design gates every such action behind role authorisation, declared engagement scope and explicit human approval for the most sensitive flows, and restricts targets to operator-declared private ranges [15]. Responsible-AI practice is observed by treating the model as an untrusted proposition generator whose output is validated deterministically and reported honestly (Sections 3.5 and 3.6); capability, limitation and residual failure modes are published rather than suppressed. Accessibility is respected at the interface level (clear, structured API responses rather than prose), and because the product processes no personal data, it falls outside personal-data protection regimes; the applicable disclosures and declarations are collated in Chapter 3.6 and Appendix B.

Table 4.2: Verified pipeline stages and middleware under `src/`.

Component	Module	Primary responsibility
S1 InputNormalizer	`preprocessing/input_normalizer.py`	NFKC normalisation, case folding, jargon standardisation
S2 AliasResolver	`preprocessing/alias_resolver.py`	Alias-to-canonical (CVE, tool, action) resolution
S3 OllamaInference	`inference/ollama_inference_engine.py`	Local LLM inference with strict-JSON directive
S4 JSONParser	`validation/json_parser.py`	JSON extraction; strips fences and thinking tags
S5 SchemaValidator	`validation/schema_validator.py`	Typed `IntentSchema` enforcement
S6 RegexValidator	`validation/regex_validator.py`	IP / CIDR / CVE / port pattern verification
S6B NetworkValidator	`validation/network_validator.py`	Scope containment, CIDR sanity, target-type consistency
S7 ScopeGuard	`validation/scope_guard.py`	RFC 1918 enforcement and injection re-scan
M1 AdversarialDetector	`middleware/adversarial_detector.py`	Three-layer threat scoring, block/flag before inference
M2/M5 RbacGuard	`middleware/rbac_guard.py`	Pre-inference fast-fail and post-inference intent check
M3 SessionContext	`middleware/session_context.py`	Multi-turn context, last-target carry-over, TTL expiry
M4 SemanticCache	`middleware/semantic_cache.py`	Exact-hash and embedding-similarity cache hit
Sub-intent classifier	`middleware/sub_intent_classifier.py`	Refines intent into hierarchical sub-intents
ContractBuilder	`pipeline/contract_builder.py`	Synthesises the downstream `PlannerContract`

5. IMPLEMENTATION, INTEGRATION AND TESTING

5.1 Development Environment and Technology Stack

The IRE is implemented as a modular Python service with a FastAPI interface and a clean internal separation between the core stages (S1–S7), the guard middleware (M1–M5) and the language-model adapter (Chapter 4). This section records the verified technology stack and the development environment in which the system was built and measured.

Runtime and deployment platform

The service is written in Python 3 and runs as a local daemon under Uvicorn on port 8001. The development launcher (`scripts/run_dev.py`) targets Python 3.11+ and performs pre-flight checks for the `.env` file, Ollama connectivity on port 11434, the configured model's availability and the development server. The evaluation and QA environments for this dissertation were exercised under Python 3.9.13 (the retained virtual environment), which is below the recommended version but within the declared compatibility window. Table 5.1 lists the verified stack, pinned where a version is recorded in `requirements.txt`.

Table 5.1: Verified technology stack.

Layer	Technology	Purpose
Language	Python 3.11 (3.9.13 used)	Implementation platform
API framework	FastAPI 0.115.0, Uvicorn 0.30.6	HTTP surface, ASGI server
Configuration	pydantic-settings 2.4.0, python-dotenv	`.env`-driven settings
Schemas	Pydantic v2 ($\geq$ 2.9, $<$ 3)	Typed model validation
Inference client	ollama 0.3.3, httpx 0.27.2	Local Ollama decoding
Rate limiting	slowapi 0.1.9	30 requests/min per caller
Logging	structlog 24.4.0	Structured JSON audit
Embeddings	sentence-transformers 3.0.1	Cache similarity, consistency
Testing	pytest 8.3.2, pytest-asyncio 0.23.8	Unit + async integration

Two declared dependencies are flagged for accuracy: `spacy` 3.7.6 is listed in `requirements.txt` and probed by the launcher, and `jsonschema` 4.23.0 is listed in the SRS, but the implemented validation paths in `src/` use the Pydantic

runtime, not these packages. They are treated in this dissertation as declared-but-unused rather than as part of the runtime (Chapter 4).

Source layout and configuration

The service is organised under `src/`: `api`, `audit`, `inference`, `integration`, `middleware`, `pipeline`, `preprocessing`, `schemas`, `utils` and `validation`. Central configuration lives in `config/settings.py` with the feature, RBAC, and enforcement policies in `features.yaml`, `feature_flags.py`, `rbac_policy.py` and `enforcement_policy.py`. Runtime constants are not decided by the code: they are loaded from the environment and the YAML feature flags at startup, so every experimental constant is reproducible from the committed configuration (Section 3.7).

5.2 Component Implementation

This section describes how the design of Chapter 4 is realised in code. The descriptions are based on the implementing modules under `src/` and `config/` as committed.

Pipeline orchestration

`src/pipeline/ire_pipeline.py` instantiates every stage and middleware and defines their execution order. Each middleware and stage is wrapped in its own exception boundary so that a failure in non-critical middleware degrades gracefully instead of aborting the request (Section 4.1). The order is: M1 adversarial scan, M2 pre-inference RBAC, M3 session-context injection, M4 semantic-cache lookup, then the core S1–S7 with S6B, then M5 post-inference RBAC, cache store, v2-response assembly, sub-intent enrichment, session update and session-info enrichment. Every stage and middleware that can reject a request does so with a classified error carrying the stage name, the divergence field and its accumulated `ValidationFinding` entries.

Core stages

- **S1 InputNormalizer** (`preprocessing/input_normalizer.py`) applies NFKC Unicode normalisation, strips control characters and thinking tags, collapses whitespace and standardises a jargon table (for example `syn scan` → `SYN stealth scan`, `pwn` → `exploit`).

- **S2 AliasResolver** (`preprocessing/alias_resolver.py`) resolves a 22-entry `data/alias_map.json` of exploit surnames, slang and tool terms to canonical identifiers before inference.
- **S3 OllamaInferenceEngine** calls the local model with a few-shot, strict-JSON directive, a per-stage timeout (60 s), a 2048-token cap and a small response cache (`inference/ollama_inference_engine.py`).
- **S4 JSONParser** (`validation/json_parser.py`) recovers the JSON payload, stripping code fences, markdown artefacts and hard bounds on extraction.
- **S5 SchemaValidator** (`validation/schema_validator.py`) enforces the typed `IntentSchema` with Pydantic and maps failures to hallucination classes (fabricated CVE, target-type mismatch, ungrounded confidence, fabricated parameter).
- **S6 RegexValidator** re-verifies IP/CIDR/CVE/port/URL entities and rejects shell metacharacters (`validation/regex_validator.py`).
- **S6B NetworkArchitectureValidator** checks engagement-scope containment, CIDR sanity (widest prefix 30, narrowest-32 scan targets) and target-type consistency (`validation/network_validator.py`).
- **S7 ScopeGuard** (`validation/scope_guard.py`) enforces RFC 1918 membership [15], the engagement mode (strict/warn/disabled) and re-scans the raw input as a final injection gate.

Guard middleware

- **M1 AdversarialDetector** (`middleware/adversarial_detector.py`) implements the three-layer detector of the design: a lexical layer scoring known injection phrases and control tokens, a structural layer scoring encoded payloads and script-like constructs, and a scoring layer that auto-blocks at a cumulative score of at least 0.95 and flags at 0.7.
- **M2/M5 RbacGuard** is driven by `config/rbac_policy.py`, which declares the role hierarchy (viewer, analyst, operator, admin), the intent-permission matrix, the three sensitive intents and the restricted-role set (`viewer`); unknown roles coerce to `viewer`.
- **M3 SessionContextStore** (`middleware/session_context.py`) tracks per-session turn history with a TTL, injects a memory window and carries the last target forward.
- **M4 SemanticCache** (`middleware/semantic_cache.py`) serves exact-hash and embedding-similarity hits and is invalidated with the CVE grounding.

- **sub-intent classifier** (`middleware/sub_intent_classifier.py`) enriches a valid response with a hierarchical sub-intent that drives tool selection.

Contract generation, audit and metrics

The downstream `PlannerContract` is synthesised by `src/pipeline/contract_builder.py`: sub-intent selects the tool (for example `NETWORK_SCAN.SYN_STEALTH` → `nmap -sS`), modifiers map to flags, priorities are assigned, and exploitation flows are marked for human approval. The audit trail in `audit/audit_logger.py` persists every finding to an append-only SQLite database (`data/audit_log.db`), hashing operator content (SHA-256) while keeping intent, role, latency and outcome as structured fields. `utils/metrics_collector.py` records latency percentiles, per-stage error counts and intent distribution for the `/metrics` endpoint.

5.3 Integration with the Other Components

The IRE is Component 1 of the NeuroShell framework and integrates with Components 2–4 under feature flags, so the framework can be turned on without changing the parsing behaviour. `src/integration/planner_client.py` forwards validated contracts to the Dynamic Planner (Component 2, port 8002) when `planner_enabled` is true; `src/integration/executor_client.py` targets the AEERE executor (Component 3, port 8003) and the AVAE vulnerability analysis (Component 4, port 8004) under their respective flags. Clients authenticate with a shared API key over HTTP and a bounded timeout.

Two integration modes are implemented in the API surface. `POST /parse` returns the validated intent and pushes a background task to the planner when a contract is actionable; `POST /execute` runs the synchronous C1-to-C2 chain and returns both outputs, persisted in `utils/chain_cache.py` so an identical input short-circuits both services. `POST /parse/plan` returns the planner-ready contract directly. Downstream consumers receive `PlannerContract` objects only, never free-form model output; the Component 2 safety gates (RFC 1918, dangerous-pattern checks) remain a second, independent enforcement point (Section 4.2).

The full API surface, implemented in `src/api/main.py`, is:

- GET `/health`;
- GET `/metrics`;

- POST `/parse`;
- POST `/parse/plan`;
- POST `/execute`;
- GET `/session/{id}` and DELETE `/session/{id}`;
- GET `/admin/rbac/{role}`;
- GET `/admin/features` and POST `/admin/features/reload`;
- GET `/admin/models` and POST `/admin/models/switch`;
- GET `/admin/enforcement-policy`;
- GET `/admin/escalations` and POST `/admin/escalations/{id}/resolve`;
- GET `/admin/audit/summary`;
- GET `/admin/chain-cache` and DELETE `/admin/chain-cache`;
- GET `/engagement/scope`.

Requests are authenticated with an API key and rate-limited to 30 per minute per caller; command length is capped in the API.

5.4 Verification and Software Testing

Strategy

The verification strategy distinguishes unit-level determinism from integration-level wiring. Unit tests exercise every guard path, every cell of the RBAC matrix, both semantic-cache hit and miss paths, all six hallucination classes, and the metrics calculations, using a live-free harness in which the LLM is mocked so the suite is deterministic (no live model dependency in the unit or integration paths). Integration tests exercise the HTTP surface with an `httpx.AsyncClient`: the asynchronous parse flow, the adversarial-to- alias sequence, the full-stack guarded pipeline, and the audit and escalation queue lifecycle. The suite is the gate that must pass before any measurement in the results chapter is accepted (Section 3.7).

Re-derived results

The figures in this section were re-derived by running the committed suite directly for this dissertation (session 2026-10-07), because the retained coverage artefact was internally inconsistent with the current source (Chapter 3).

- **Suite.** 506 collected tests, 506 passed, 0 failed in the full run (111.76 s; 49.7–55.3 s across runs). The split is 434 unit tests and 72 integration/async tests, an increase

from the 482 tests (434 unit + 48 integration) documented on 2026-08-26 because additional integration cases were added to the API, full-stack and audit suites.

- **Coverage.** 76% statement coverage across `src/` (2421 statements, 582 missed), with 17 modules at 100%. The safety-critical core is well covered: adversarial detection 99%, schema validation 99%, session context 96%, audit logger 94%, semantic cache 94%, sub-intent classifier 89%. The weakest areas are the integration clients (planner 30%, executor 16%), `contract_builder.py` (30%) and `chain_cache.py` (24%), which are orchestration paths rather than safety invariants.
- **Static analysis.** `ruff` reports 593 findings, 100% style and modernisation, with no correctness-level (E9) errors and no fatal findings.

Evaluation harness

The evaluation harness applies the procedure of Section 3.3 to the golden data set, computes the metrics of Section 3.4, and regenerates every table used in Chapter 6 from a single, manifest-pinned command (Section 3.7). The harness records per-record decisions so the reported figures can be reproduced without re-invoking the model, and it asserts the gate condition (suite green) before accepting a measurement.

5.5 Deployment and Operation

Development deployment

A single command (`python scripts/run_dev.py`) performs the pre-flight checks and starts Uvicorn on port 8001 with hot-reload for development. The service binds to 0.0.0.0 for the downstream components on the device; no external service is contacted. Health and readiness are exposed at `/health`, which reports Ollama connectivity, the active model, the seven-stage pipeline and the middleware enablement map.

Containerised deployment

A `Dockerfile` provides a reproducible deployment on Python 3.11-slim, installing the pinned requirements and packaging `src/`, `config/` and the alias map; a `docker-compose.yml` (documented in the SRS) orchestrates the service alongside the Ollama runtime, and the image exposes a health check against `/health`. The

container runs a single worker by design, because the in-memory cache and session store are per-process state (Section 4.2).

Operational administration

The admin surface supports runtime reconfiguration without redeploys: model listing and switching, feature-flag inspection and hot reload, live enforcement-policy summaries, escalation resolution, audit-trail summaries and engagement-scope inspection. Rate limiting and command-length caps protect the model backend, and session/cache storage can be inspected and flushed through the API. Deployment properties are traceable to the trust boundary described in Chapter 4 and the threat-model treatment of Chapter 3.

6. RESULTS AND VALIDATION

This chapter reports the measured outcomes of the study. Every figure is a point estimate computed from the committed experiment record by the reproduction command of Section 3.7, using the metrics of Section 3.4 and the procedure of Section 3.3. All measurements are single-run: no confidence intervals are attached, and the comparison rule of Section 3.5 (a difference outside the ±1–2 pp recorded run-to-run noise band is read as a layer effect) applies throughout.

6.1 Research Results

6.1.1 Experiment 1: Pipeline Value (RQ1)

Raw, single-shot model output performs no flaw classification, so it trivially achieves a 0% catch rate on all 900 flawed records, and it misclassifies the intent of 22% of the 600 well-formed records with Gemma and 31% with Qwen. Adding the guarded pipeline raises decision accuracy D_{Acc} (Equation 3.5) from 78% to 86% for Gemma and from 69% to 82% for Qwen, raises the flaw-catch rate (Equation 3.6) from 0% to 72% for Gemma and to 64% for Qwen, and reduces the false-positive rate (Equation 3.7) from 32% to 14% for Gemma and from 31% to 15% for Qwen (Table 6.1). The largest improvement is in the catch rate, by construction: raw output is never flagged at all.

Table 6.1: Experiment 1: raw model output against the guarded pipeline on the full 1500-record set (N=1500, D^+=600, D^-=900).

Configuration	D_{Acc}%	Catch%	FP%
Gemma raw	78	0	32
Gemma guarded	86	72	14
Qwen raw	69	0	31
Qwen guarded	82	64	15

The result answers RQ1 directly: the guardrail stack converts output that carries no machine-checkable safety property into output whose acceptance is conditional on a verified conjunction of predicates (Section 3.1). The catch-rate gap is categorical rather than incremental, because raw output has zero catch by construction.

6.1.2 Experiment 2: Backend Head-to-Head (RQ3)

Under the identical guardrail stack, Gemma leads Qwen on decision accuracy (86% against 82%) and on catch rate (72% against 64%) with comparable false-positive rates (14% against 15%, Table 6.2). The guarded architecture narrows the backend gap on accuracy from 9 percentage points (78% against 69% raw) to 4 points, and holds the catch-gap at 8 points.

Table 6.2: Experiment 2: identical guardrail stack, both backends.

Backend	D_{Acc}%	Catch%	FP%
Gemma (guarded)	86	72	14
Qwen (guarded)	82	64	15

6.1.3 Experiment 3: Layer Ablation (RQ2 and RQ3)

Table 6.3 reports the eight configurations on the full 1500-record set for both backends, with ΔCatch relative to the fully guarded baseline (Equation 3.3). Disabling the complete deterministic validation stack (S5–S7) has the largest impact on catch rate: 30 points for Gemma (72% to 42%) and 24 points for Qwen (64% to 40%). Among individually ablated layers, the hallucination validator contributes most to catch rate (9 points for Gemma, 14 points for Qwen). Its accuracy effect differs by backend: largest for Qwen, whereas the scope guard has the largest accuracy effect for Gemma.

Removing RBAC reduces both accuracy and catch in both backends while raising FP, indicating that the role check improves accuracy and lowers false positives simultaneously. The network validator is the second-largest accuracy contributor (18 points Gemma, 24 points Qwen). The scope guard, network validator, self-consistency gate and semantic cache each show a *rise* in catch on removal (2–10 points) together with a sharp rise in FP (to 30–44%). Per the reading rule of Section 3.5, these positive deltas signal interactions rather than contributions: the layers act as false-positive suppressors, and the pre-emptive-rejection hypothesis is recorded as an open mechanism (Section 7).

6.1.4 Experiment 4: Per-Class Catch (RQ2)

On the dedicated deep-dive subset, catch is strongest exactly where a deterministic rule exists (Table 6.4). Adversarial injection is caught at 95.6% by the lexical and structural

Table 6.3: Experiment 3: ablation on the full 1500-record set. ΔCatch is in percentage points relative to the fully guarded pipeline (a negative value means the layer contributes); Noise is the approximate run-to-run variation in percentage points. The validation-stack row is the joint S5–S7 ablation.

	Gemma				Qwen				
Configuration	D_{Acc}	Catch	FP	ΔCatch	D_{Acc}	Catch	FP	ΔCatch	Noise
Full pipeline (baseline)	86.0	72	14	–	82.0	64	15	–	±1–2
No scope guard	65.0	74	42	+2	68.0	68	30	+4	±1–2
No RBAC guard	75.0	65	20	−7	57.0	57	17	−7	±1–2
No network validator	68.0	80	40	+8	58.0	72	32	+8	±1–2
No halluc. validator	66.0	63	40	−9	52.6	50	34	−14	±1–2
No validation stack	65.0	42	15	−30	62.0	40	16	−24	±1–2
No self-consistency	68.0	78	42	+6	58.0	74	36	+10	±1–2
No semantic cache	67.0	77	44	+5	60.0	70	40	+6	±1–2

layers of the detector, and RBAC violations at 100% by the deterministic pre-inference role check. Fabricated parameters (87.5%) and type mismatches (81.3%) are detected by the S5–S6 syntactic rules, and fabricated CVEs (78.1%) by the hallucination validator's offline grounding. The residual weak classes are the semantic cases—contradictory requests (59.4%), out-of-scope requests (46.7%) and ambiguous requests (40.0%)—where no deterministic signal exists and the pipeline relies on the self-consistency gate. In the full set these three classes account for 339 of the 900 flawed records (37.7%). Per-class denominators therefore carry the interpretation, as fixed in Section 3.5.

Table 6.4: Experiment 4: per-class catch rate and primary detection mechanism (deep-dive subset, each class's own denominator).

Flaw class	Catch%	Mechanism
RBAC violation	100.0	M2 role check
Adversarial inj.	95.6	M1 detector
Fabricated param.	87.5	S5–S6 rules
Type mismatch	81.3	S5–S6 rules
Fabricated CVE	78.1	hallucination validator
Contradictory	59.4	self-consistency
Out-of-scope	46.7	self-consistency
Ambiguous	40.0	self-consistency

6.2 Product and Technical Results

6.2.1 Verification and testing

The automated suite was re-derived by direct execution for this dissertation (session 2026-10-07; Chapter 5): 506 collected tests, 506 passed, with 434 unit and 72 integration tests, and 76% statement coverage across `src/` (2421 statements, 582 missed; 17 modules at 100%). The safety-critical core is the best-covered part of the system: adversarial detection, intention schema, RBAC, scope guard and the regex and network validators each sit at or above 93%, while audit logging, semantic cache and session context sit at 89–96%. Static analysis (`ruff`) reports 593 findings, all style and modernisation with no correctness-level (E9) errors and no fatal findings. These results are the product-level acceptance gate: they support the empirical measurements of Section 6.1 being representative of the committed system.

6.2.2 Known limitation achieved

The three semantic classes with the lowest catch in Experiment 4 (ambiguous, out-of-scope, contradictory) are exactly the classes without a deterministic signal; their residual misses constitute the principal functional gap of the component and are analysed in Chapter 7.

6.3 User or Stakeholder Validation

The study involves no human participants (Section 3.6), so no usability or user-preference study was conducted. Stakeholder validation is instead evidence-based, through two channels. First, the verification gate of Section 6.2—a deterministic, live-free test suite that must pass before any measurement is accepted—validates the component against its stated requirements (Section 4.1) and the invariant of Section 3.1. Second, the NeuroShell framework integration review (Section 6.4) validated the component's contract against Component 2 of the framework on the receiving side.

6.4 Integration Results

The C1-to-C2 integration analysis (session 2026-08-28; documented in the project record) verified the contract hand-off against the real Component 2 engine at port 8002. The 9-value intent literal sets match exactly, and the flat `PlannerContract` fields map 1:1 onto the Component 2 request. Live checks confirmed: a well-formed in-scope

contract returns a structured planner output with an empty safety block; a rejected intent is refused by Component 2 with its filter-stage error; an out-of-scope public target is rejected by the Component 2 RFC-1918 gate; and a wrong API key is refused with 401. These observations confirm that the guarded IRE contract carries its integrity through the downstream component, so the safety invariant of Section 3.1 remains meaningful at the framework boundary. **[TODO: The C1-to-C2 live verification used the available stand-in model at the time; re-run the two-element happy-path check against the production *gemma4:e4b* backend once the model is provisioned on the evaluation machine, and record the result in the committed experiment record.]**

6.5 Summary of Findings

The measured outcomes answer the three research questions as follows. RQ1: the guarded pipeline raises decision accuracy by 8–13 points, the catch rate from 0% to 64–72%, and lowers the false-positive rate to 14–15% over raw output. RQ2: the deterministic validation stack is the dominant safety contributor (24–30 catch points), the hallucination validator is the dominant single layer (9–14 catch points), and the semantic classes remain the weakest because they lack a deterministic signal. RQ3: the catch-rate ordering of layers carries across both backends, but the accuracy contribution of individual layers is backend-specific, so cross-backend generality claims apply to catch ordering rather than to a single accuracy ranking. The quantitative case is made by Table 6.1, Table 6.2, Table 6.3 and Table 6.4, and is interpreted in Chapter 7.

7. DISCUSSION

This chapter interprets the measurements reported in Chapter 6 against the research questions of Section 1.6, compares the guardrail stack with the surveyed baselines of Chapter 2, and separates the research from the product contribution. It then states the verified limitations and threats to validity and draws the practical and research implications.

7.1 Interpretation of Findings

The guarded pipeline answers the central question affirmatively: a small, offline, unattended language model can be made reliable enough to gate an automated security copilot, provided its output never reaches downstream tooling before it satisfies a conjunction of deterministic predicates. The measured basis for this claim is the experiment record reported in Chapter 6, whose headline deltas are considered under each research question in turn.

RQ1: pipeline value

The raw model is not merely imprecise—it is unusable as an intent classifier. Experiment 1 (Section 6.1.1) shows a catch rate of 0% on both backends: a single-shot schema probe never happens to produce a flaw-free intent for the nine-category taxonomy, and the false-positive rate compounds the problem—32% for Gemma and 31% for Qwen of well-formed records are rejected or misclassified. The guarded pipeline converts this into D_{Acc} 86%/82%, catch 72%/64% and FP 14%/15% (Gemma/Qwen, Table 6.1). The accuracy gain is therefore not the headline result: it is the catch rate, which the guards raise from zero to between two thirds and three quarters of all flawed records while simultaneously halving the false-positive rate. The most useful single reading of RQ1 is the asymmetry between the two movements—the deterministic stack adds the ability to detect flaws, and reduces spurious rejection at the same time, rather than trading one for the other.

RQ2: layer contribution

Experiment 3 (Table 6.3) identifies the validation stack as the decisive component: removing it entirely collapses catch from 72% to 42% for Gemma and 64% to 40% for Qwen (ΔCatch -30 and -24). Individually, the hallucination validators are the

largest single-layer contributors to catch (−9 Gemma, −14 Qwen), which is consistent with the dominant flaw classes in the synthetic population: fabricated parameters and CVEs are the most frequent flaw kinds. The design insight from Experiment 4 (Section 6.1.4) is that per-class catch accords with determinism: the classes that expose a syntactic or enumerable signal (RBAC violations, adversarial phrasing, fabricated parameters and CVE identifiers) are caught at 78–100%, whereas the classes that require meaning resolution (contradictory, out-of-scope, ambiguous) remain at 40–59%. RQ2 is therefore answered both at the layer level and at the class level: the guard layers that contribute most are exactly the ones that encode decidable predicates, and the residual risk concentrates in the predicates that are not decidable.

The negative ΔCatch entries in the left column of Table 6.3—removing the ablation-row guard apparently raising catch—are read, as Section 3.5 commits, not as a deficiency of running the counterfactual treatment, but as a removal-adversity finding: the default configuration trades a small portion of catch against a large preservation of usability (false positives and accuracy). This is expected under the framework of Chapter 2 and does not contradict the safety invariant; it is artefact behaviour of the committed single-run protocol rather than evidence about the production setting.

RQ3: generalisability

The catch-rate ordering of the ablation rows is broadly consistent across the two backends (Experiment 3), so RQ3 is answered affirmatively for the safety dimension: deterministic guardrailing, not model choice, is the primary driver of catch-rate performance, provided the trust boundary remains closed. The condition on that conclusion is the accuracy dimension. The accuracy contribution of individual layers is backend-specific—the scope guard dominates accuracy for Gemma, the hallucination validator for Qwen—so the generality claim must be restricted to the ordering of catch-rate effects (Section 3.5) and explicitly must not be extended to a claim that any single layer is required in the same rank order to preserve accuracy across backends. The measured safety therefore generalises; the measured usability-tuning does not.

7.2 Comparison with Literature Baselines and Existing Solutions

Against the existing-systems survey (Table 2.1), the IRE occupies the combination that the review found empty: a dedicated intent-recognition layer that is machine-consumed, schema-checked, scope- and authorisation-enforced, offline and CPU-only,

in the single-inference budget. The comparison with each family makes the positioning concrete.

The generation-side copilots—PENTESTGPT [1] and the sparring-partner system [2]—demonstrate that a model can drive security tooling, but both route output to a human reviewer and validate correctness informally. The IRE's checked output is structurally different: it is a machine-consumed intent contract that has already passed deterministic validation before any executor sees it. The measured 72%/64% catch rate does not compete with those systems' task coverage; it answers a question they do not pose, namely what fraction of flawed machine-bound decisions is caught before dispatch.

Fabrication-detection and agreement methods—SelfCheckGPT [7], semantic-uncertainty and semantic-entropy approaches [8, 9], and self-consistency decoding [10]—assume repeated or interactive generation whose cost collides with the offline, CPU-only, single-inference budget adopted here (Section 2.2). The IRE does not abandon their rationale; it relocates it. Instead of sampling a verifier, it resolves the cost tension with a bounded-sampling agreement gate and a deterministic escalation path (the hallucination validator) that needs no large model (Section 2.3). The measured greed of the semantic cache (Section 6.1.3) is the accuracy-pressure relief that makes this resolution practical on the constrained device.

The comparison with OWASP-prioritised defences [6] and the empirically demonstrated insufficiency of individual prevention techniques [4, 5, 16] is the strongest one. Those surveyed defences act as post-hoc application-layer checks; the IRE's contribution is that the checks are composed into a stage that sits *between* generation and dispatch, where each predicate (schema, syntax, network, scope, authorisation) is independently decidable on a closed intent taxonomy. The ablation evidence that the complete stack outperforms any single layer is the empirical counterpart of that architectural difference, and it is the reason the comparison proceeds at the level of a composed system rather than of individual techniques.

7.3 Research Contribution

The research contribution is the answered question, not the artefact. Three stated findings constitute it.

First, the deterministic validation stack is the dominant driver of the measured safety, and its effect is reproducible across the two backends. The ablation evidence in Section 6.1.3

shows the stack removing $30/24$ catch points against a $9/14$ contribution from the largest single layer; the same ordering holds across backends. This generaliates the narrow claim "guardrails help" into a measurable ordering result: which layers matter, and in which combination.

Second, the effectiveness boundary is predictable: catch concentrates on flaw classes that expose decidable structure. The per-class catch spectrum of Section 6.1.4 runs from 100% down to 40%, and the ranking is exactly the decidability ranking developed in the conceptual framework (Section 2.5). This turns the limitation of single-shot offline inference into a design law for trustworthy automation: a model in the closed-intent setting should be asked for structure, never for verdicts.

Third, usability and safety can improve together. The guarded pipeline simultaneously raises accuracy, raises catch and halves false positives (Section 6.1.1). The experiment record therefore weakens the frequent implicit assumption that adding deterministic checks to model output must trade either precision for recall or fluency for utility.

7.4 Product Contribution

The product contribution is the shippable IRE implementation developed in Chapter 5 and exercised by the integration analysis of Section 6.4: a guarded intent-recognition engine that converts free-form offensive-security instructions into validated, schema-constrained, authorised machine contracts end-to-end. Its measured and verifiable properties are an automated test suite of 506 tests (76% statement coverage, 434 unit and 72 integration) that encode the safety invariant as executable expectations, and a live C1–C2 contract verification that exercised the real code paths (Section 6.4).

The product's differentiated value sits in the stack, not in the model: the engine's own documentation and configuration define OWASP-aligned guardrail hardening, and the safety gate rejects or warns on every flaw class rather than rendering model output downstream. Within the deployment constraints– offline, CPU-only, single-inference, retrieval-free–the product is the only surveyed design in Table 2.1 that provides the full conjunction of schema check, scope/authorisation enforcement and offline operation.

The practical caveats are the ones verified in the source audit (Section 4.1 and the integration record): several documented dependencies are not implemented (flagged as *doc-only*); the shipped configuration uses a default development API key that bypasses transport authentication when left untouched; and the "`production`" scope-mode value admitted by the settings schema is dead configuration that falls through to the

`WARN` enforcement path rather than `BLOCK`. These are real product-integrity gaps and are stated as such; none affects the validity of the evaluation, which ran with production credentials and strict scope semantics, but each must be closed before the engine is deployed in an untrusted environment.

7.5 Limitations and Threats to Validity

The limitations are stated precisely because the dissertation commits to evidence over assertion. They are organised by validity category and, where possible, bounded by the reproducibility record of Section 3.7.

Single-run point estimates and the absence of confidence intervals are the principal internal-validity limitation. Each configuration in Experiments 1–4 was executed once (Section 3.3), so no measurement error or seed sweep bounds the deltas; the magnitude of individual layer effects, especially the small negative-ΔCatch entries, must be read as one-sample observations. The protocol mitigates this by committing the execution in advance and fixing the data set, but it does not remove it.

Synthetic, non-uniform data bounds content validity. The 1,500-record population is generated against a committed taxonomy, and its category sizes are non-uniform (600 D+ / 900 D-, and per-class sizes that differ), so accuracy and catch figures are population-specific and should not be read as population-mean estimates for real production traffic.

Closed threat boundary bounds external validity. The design explicitly excludes retrieval and free-form rendering, which truncates the indirect- and tool-injection channels, but the evaluation does not cover threats outside the trust boundary: indirect prompt injection, physical device compromise, or backdoored model weights were never presented to the engine. The claim is therefore "reliable within the evaluated threat model", not "reliable unconditionally".

Hardware-dependence. All experiments ran on a single CPU-only device, so conclusions about throughput, cache behaviour and feasibility are single-platform observations. The hardware bound is recorded in the reproducibility manifest (Section 3.7).

Unconfirmed mechanism. The mechanism behind the increase-on-removal pattern of the ablation's negative-ΔCatch rows is unconfirmed; it is attributed to removal-adversity interactions but no mechanistic study isolates it. This is flagged as open rather than explained.

Weak protection of semantic classes. The three semantic flaw classes (contradictory, out-of-scope, ambiguous) are caught at 40–59% because no deterministic signal exists for them. The research question of how to improve these classes within the offline budget is left open.

Engineering configuration caveats. As stated in Section 7.4, the default development API key and the dead "`production`" scope mode are verified findings that materially affect deployment posture and are recorded as product limitations, not conflated with the evaluation.

Latency, energy and operational cost are acknowledged as unmeasured throughout. The evaluation prioritised correctness and catch; an operator deploying the engine should measure end-to-end latency on their own hardware before committing to interactive use.

7.6 Implications

Implications for practice

For practitioners deploying language models in security automation, the strongest implication is architectural: put a deterministic, decidable validation stage between a small offline model and machine dispatch, and the safety and usability of the system can improve together rather than trade. The catch-before-dispatch posture means the model is consulted for structure while the system decides; this is directly actionable with offline inference and a closed intent taxonomy, and it does not require a large verifier model. The verified product gaps—default development credentials and the dead `production` scope route—should be treated as design requirements for any similar engine before deployment.

Implications for research

The measured ordering result answers RQ3's conditional reading: safety generalises across backends, accuracy tuning does not. That split suggests the research frontier is not "which model" but "which decidable predicate structure", and it strengthens the case for composing model behaviour with enumerable checks over sampling-based agreement for single-inference budgets. The weak-protection gap for the three semantic classes is the clearest open problem: closing it within the offline budget (for example with stronger structured prompting, targeted enumerators, or the bounded-sampling escalator) is a concrete direction for future work. The single-run limitation should be

removed by repeating the committed protocol with seed sweeps and interval estimates to convert the point estimates of this dissertation into bounded ones.

Implications for the thesis

For the dissertation itself, the implication is that the central question is answered with the evidence available and the gaps are stated with the sources that would close them. The evaluation remains valid within the closed threat model and the committed protocol; the follow-up work—production provision of `gemma4:e4b` for the C1–C2 happy-path re-run recorded in Section 6.4, latency measurement, and the semantic class research—is itemised in the project's reproducibility record rather than asserted.

8. COMMERCIALISATION AND SUSTAINABILITY

This chapter examines how the Intent Recognition Engine could be taken from a research artefact to a deployed capability. It states the value proposition and the users it would serve, positions the engine against the surveyed landscape, and sets out plausible—not claimed—commercialisation and sustainability pathways. Consistent with the evidence-first rule of this dissertation, no market statistic is fabricated here, no revenue or adoption figure is asserted, and every legal or certification statement is either grounded in the verified record of the preceding chapters or explicitly marked as a requirement rather than an achievement.

8.1 Value Proposition and Target Users

The value proposition follows directly from the measured result of Chapter 6. A security organisation that wants to let analysts drive tooling through natural language faces a choice between correctness and automation: a raw language model offers fluency but, as Experiment 1 shows, transfers every malformed, misclassified or hallucinated output into the actions a pipeline would execute—a 0% catch rate and a 32%/31% false-positive rate before guarding. The IRE converts that choice into a combination: on the evaluated 1,500-record population, the guarded pipeline raises flaw catch from 0% to 64–72%, lifts decision accuracy from 69–78% to 82–86%, and halves the false-positive rate (14%/15%), while remaining offline, CPU-only, single-inference and retrieval-free (Section 6.1.1). The value is therefore not "an LLM that understands security prose"—it is "a head-of-pipeline component that guarantees structurally valid, schema-constrained, authorised and in-scope intent contracts despite the underlying model".

This value concentrates where three conditions coincide: the organisation uses offensive-security tooling, operates under data- or policy-constraints that disallow sending prompts to a hosted model or a retrieval service, and has an automation chain that can consume validated JSON contracts rather than prose (Section 4.2). Three user groups follow.

Security analysts and operators are the primary users. For them the engine removes the manual translation stage between intent and tool invocation, and the safety invariant (Equation 3.2) means a misremembered tool syntax, an out-of-scope target or a role violation is caught by deterministic predicates rather than by the analyst's vigilance. The audit-friendly structured response and per-request metrics (Section 5.5) also suit post-action review.

Security-tooling integrators are the secondary users. Because the IRE emits a closed, schema-enforced contract with a documented tool hint and parameter envelope (nine intent classes, validated target types and explicit CVE/port/modifier fields), it is an integration point for orchestrators and planners that would otherwise re-parse prose (Section 4.2). Within the broader NeuroShell chain (project reference R26-CS-018, Section 1.1) the IRE is Component 01 and is designed to be the validated eye of the chain.

Offline-constrained environments constitute the differentiating third user group: air-gapped networks, regulated industries, embedded or edge deployments and research ranges where external inference is unavailable or prohibited. The engine's only runtime dependencies are model weights, policies and an operator API key, and nothing leaves the device at inference time (Section 5.5). This is the segment the comparable copilots in the survey do not serve (Section 2.4).

8.2 Market and Competitive Position

The competitive position is stated relative to the verified existing-systems survey (Section 2.4) rather than to unverifiable market sizing. The survey found a four-way split: deterministic tooling (for example Nmap and the exploitation framework) that is ground-truth accurate but requires a human to map intent, generation-side copilots that map intent but consume human review, integration techniques that connect models to tools without an intermediate validation stage, and post-hoc guardrail techniques that check model behaviour after the fact. In those four categories, any candidate is strong in one and absent in another: none combined machine-consumed output, schema and scope/authorisation enforcement, and offline, CPU-only, retrieval-free operation over a closed intent taxonomy (Section 2.5 and Table 2.1).

The IRE occupies that empty combination, and its defensible position rests on three properties the evaluation supports rather than on marketing claims. First, the catch-before-dispatch architecture is a behavioural claim with a measurement: the ablation record shows the validation stack is the dominant safety contributor (Gemma -30, Qwen -24 catch points when removed; Section 6.1.3). Second, the result is reproducible and backend-independent on the safety dimension—the identical stack orders the effects of the same way on two distinct offline models (Section 6.1.2)—which weakens a "vendor-locked model" objection. Third, the offline posture is structural, not incidental: it is the deployment constraint the design is built around, not a configuration.

Two competitive risks are stated openly. The large-model copilots are strong on coverage and conversation, so the IRE competes only in the automated, machine-consumed corner of the space, not against interactive assistants. And because no named commercial product was included in the survey for lack of verifiable primary sources (Section 2.4), the position here is a review-based gap claim, to be re-verified against named products at any commercialisation step before a broader claim is made.

8.3 Business and Sustainability Model

Consistent with the master rules, the following is a set of potential pathways, not a claimed business. All are sketched as options that a founder or institution could evaluate, and none asserts a market or revenue number.

Open-core with managed hardening services

The engine, its data set and its evaluation harness are research artefacts versioned with the project source (Section 3.7), and the model backends are open-weight and locally runnable. A natural model is open-core: the validated core is published to establish trust and to grow an integrator base, while the commercial layer concentrates on what a single-team paper cannot ship—the hardening and certification work itemised in Section 8.4 (bounded latency measurement, closed production scope path, credential management), integration packs for the principal scanning and exploitation tools, and a maintained contract-schema migration path.

Direct licensing and private deployment

An alternative or complement is direct licensing of the component to security product vendors and internal security platforms, under which the licensee embeds the IRE as the NLU gateway between their users and their tooling. The deliverable in this pathway is a library-plus-service with a documented contract schema and the test suite (506 tests, 76% statement coverage; Section 6.2) as the acceptance baseline the licensee can run independently.

Sustainability rather than revenue

A third reading of "sustainability" is operational, not financial. Because the system is CPU-only, runs on a single device without a hosted inference bill, and holds no external data dependency at inference time (Section 5.5), it has a low and predictable running

cost. That property is itself the sustainable-deployment argument: the engine can keep providing a security-gate capability in environments or budgets where hosted copilot licensing is not viable. Any real costing must be produced from measured latency and resource profiles, neither of which is asserted here (Section 7.5).

8.4 Scalability, Intellectual Property and Compliance

Scalability

The scalability story is deliberately honest about the current measurement. Each request is a single offline inference, so throughput on one device is bounded by the model's inference latency and the semantic-cache hit rate; no latency figures are quoted because they were not measured in this dissertation (Section 7.5). What scales by design is the deployment shape, not the single-node speed: because the component is a stateless, rate-limited API service holding no per-request state beyond the session context, more capacity is acquired by adding independent instances behind a load balancer, each with its own model weights, rather than by contending on one node (Sections 4.2 and 5.5). The semantic cache, the bounded-sampling agreement gate and the deterministic escalation path (Section 2.3) are the design choices that keep the per-request cost bounded as usage grows.

Intellectual property

The intellectual-property position rests on what was authored here and what is third-party. Authored: the seven-stage pipeline composition, the closed nine-intent taxonomy, the 22-entry alias-to-CVE resolver, the evaluation data set and harness, and the ablation methodology used to quantify layer contribution (Section 4.2 and Chapter 6). These are the defensible original elements, and any commercialisation should identify them as the owned asset (Section 7.4). Third-party: the model backends (`gemma4:e4b` and `qwen2.5-coder:7b`) are open-weight, the OWASP Top 10 for LLM applications [6], RFC 1918 [15] and the vulnerability-identifier registries [17] are external standards referenced by the design, and the scanning-tool syntaxes the engine emits follow vendor documentation [13, 14] (Section 4.5). A licensing review is therefore a requirement of any commercial step, not a completed fact: the statement here is that the original layers are separable from the third-party dependencies, not that any licence has been executed.

Compliance

The verified compliance position is narrow and comfortable. The product processes no personal data at any point: requests, session context and results remain on the device and involve no identifiable-individual information (Section 3.6), so it falls outside personal-data protection regimes as a data-processing matter (Section 4.5). The research itself required no institutional research-ethics approval, involved no human participants and contacted no live system, and holds only model weights and operator-configured policies as persistent assets (Section 3.6).

Two verified findings bound commercial deployment, and both are stated as requirements rather than resolved items. First, the shipped configuration uses a default development API key that bypasses transport authentication when left untouched, and the "`production`" scope-mode value admitted by the settings schema falls through to the `WARN` enforcement path rather than `BLOCK`—both must be closed before any deployment outside a controlled lab (Section 7.4). Second, institutional confirmation of the applicable declarations (integrity and AI-tools disclosures) is still pending at the final AUDIT step (Section 3.6). No certification (metric, standard or regulatory) is claimed as obtained; certification is positioned as a future hardening task.

8.5 Contribution to the Sustainable Development Goals

This section states the engine's relationship to the UN Sustainable Development Goals in the only form the evidence supports: a reasoned contribution narrative, with no quantified or statistical claim against the SDG indicators.

SDG 9 (Industry, Innovation and Infrastructure) is the most direct mapping. The engine is an applied-AI component for a critical-industry function, and its deployment argument—validated, offline, low-infrastructure automation of routine security workflows—speaks to building resilient cybersecurity capability without depending on large hosted-model infrastructure. The measured reduction in free-form model error before machine dispatch (Section 6.1.1) is precisely the reliability improvement that makes an innovation acceptable in infrastructure automation.

SDG 16 (Peace, Justice and Strong Institutions) is served through cyber resilience. By placing a deterministic, authorisation- and scope-enforced gate in front of security action, the engine contributes to protecting digital infrastructure and the institutions that rely on it. The dual-use treatment is part of that narrative: the system is deployment-gated and scope-enforced, its failure modes are published rather than suppressed, and its

offensive capability is bounded by the same invariants it enforces (Section 3.6), so the contribution to secure institutions is conditional on that gate.

SDG 8 (Decent Work and Economic Growth) is a secondary mapping: the system automates the error-prone manual translation of security intent into tool actions, the kind of task delegation consistent with freeing analysts for higher-value work. The claim is qualitative only—the engine changes the distribution of human effort, and its productivity effect is a matter for measured deployment, not an assertion made here.

No claim is made against SDG 4, 13 or others without a concrete connection, and no SDG indicator is cited with a number. This section is deliberately more modest than a typical sustainability claim because the master rule of this dissertation is that contributions are either measured or explicitly unmeasured.

9. CONCLUSION AND FUTURE DIRECTIONS

This chapter closes the dissertation. It restates the central conclusion, accounts for each objective against the experiment record with an explicit status, isolates the individual research and product contributions, and itemises the directions that follow from the limitations committed in Chapter 7.

9.1 Conclusion

The central question was whether a small, offline, unattended language model can be made reliable enough to drive an automated security copilot once its output is subject to deterministic validation. The measured answer is yes, within a closed threat model, on the evidence of this dissertation.

On a committed 1,500-record population, the guarded pipeline raised the flaw-catch rate from 0% to 64–72%, lifted decision accuracy from 69–78% to 82–86%, and halved the false-positive rate to 14–15% on two distinct offline backends (Section 6.1.1). The ablation record localised the cause: the deterministic validation stack is the dominant safety contributor (-30 and -24 catch points when removed), the hallucination validators are the largest single-layer contributors, and the per-class catch spectrum tracks the decidability of the underlying predicate (Sections 6.1.3 and 6.1.4). Safety generalises across backends while individual accuracy contributions do not (Section 6.1.2), which makes deterministic guardrailing—rather than model choice—the primary driver of the measured safety.

The conclusion is therefore not that the engine eliminates model error. It is that the engine converts an open problem—making a language model trustworthy—into the closed problem of classifying a single utterance under a known policy, and that the deterministic half of that reduction does most of the safety work. Three boundaries bound the claim and are restated because they are the honest frame for the conclusion: the evaluation is single-run point estimates without confidence intervals, the data set is synthetic and non-uniform, and the threat model closes at retrieval-free, no-free-form-output boundary (Sections 7.5 and 3.6).

9.2 Achievement of Objectives

The four objectives of Section 1.5 are accounted for individually below. Each status is grounded in the cited evidence; no objective is marked achieved on assertion alone.

- **Objective 1—design and implement the Intent Recognition Engine**: a component that receives free-text security requests and emits strictly schema-conformant, authorised intent contracts over a small offline language model. **Status: achieved.** The seven-stage core and five middleware components are implemented under `src/` and exercised by 506 automated tests (76% statement coverage; 434 unit and 72 integration) described in Section 5.2 and Section 6.2. The live C1–C2 contract verification confirmed that real code paths emit and consume validated, schema-conformant contracts (Section 6.4).
- **Objective 2—construct a purpose-built evaluation data set of security prompts, covering well-formed requests and deliberately flawed prompts across the flaw categories relevant in practice, each annotated with gold labels. Status: achieved.** The 1,500-record set (600 well-formed, 900 flawed) spans the eight+ flaw classes used in the experiments, is versioned with the project source, and underpins every measurement in Chapter 6 (Section 3.2).
- **Objective 3—establish an evaluation protocol that quantifies the pipeline improvement over raw output and abates each guard layer independently. Status: achieved.** The protocol is committed in advance, applied to the fixed data set, and recorded in a manifest for independent audit (Section 3.7). Experiments 1 and 3 operationalise the raw-versus-guarded comparison and the per-layer ablation respectively (Sections 6.1.1 and 6.1.3).
- **Objective 4—compare two offline language-model backends under the identical guardrail stack. Status: achieved.** Both backends were evaluated under the identical stack; the safety ordering was consistent across backends while per-layer accuracy contributions differed, a split recorded in Sections 6.1.2 and 6.1.3 and analysed against RQ3 in Section 7.1.

All four objectives are thus achieved. Because the evaluation is single-run and the three semantic flaw classes remain weakly protected, achievement is on the evidence available within the scope committed in Section 1.7; the boundary conditions that would warrant a "partially achieved" qualification are the same boundaries the limitations section records (Section 7.5).

9.3 Individual Research and Product Contributions

Research contributions

The research contribution is a measured result with three parts. First, the deterministic validation stack is shown to be the dominant driver of the measured safety and is reproducible across two backends: no single layer, and no model, produces the catch rate the composed stack produces (Section 6.1.3). Second, the catch effectiveness boundary is predictable from decidability: per-class catch runs from 100% on structurally decidable classes down to 40% on meaning-dependent classes, which converts the limitation of single-shot offline inference into a design law—ask a model for structure, never for a verdict (Section 6.1.4). Third, safety and usability improve together: the guarded pipeline raises accuracy, raises catch and halves false positives simultaneously (Section 6.1.1), weakening the implicit assumption that deterministic checks must trade precision for recall or fluency for utility.

Product contributions

The product contribution is the IRE itself: a shippable, deployment-gated, offline component whose output is machine-consumed, not reviewed—the empty corner of the existing-systems survey (Section 2.4 and Table 2.1). The product properties most worth exporting elsewhere are architectural: the safety invariant (Equation 3.2), the audit-friendly structured response, and the executable test suite of 506 tests that encodes the invariant (Section 6.2). The verified integration findings (Section 6.4) confirm that when the engine is asked to produce a contract for a downstream component, the contract it produces is validated, in-scope and authorised on the code paths that would run in production.

Both contributions remain subject to the honest engineering caveats itemised in Section 7.4: several documented dependencies are doc-only, the default development API key bypasses transport authentication if left untouched, and the `production` scope-mode value is dead configuration that falls through to `WARN`. These are stated as product-integrity gaps to close, and they do not affect the validity of the evaluation, which ran with production credentials and strict scope semantics.

9.4 Future Directions

The future directions follow directly from the limitations and open items committed in Chapters 6 and 7. Each is listed with the state of the evidence that motivates it.

Remove the single-run limitation. Every configuration in Experiments 1–4 was executed once (Section 3.3). Repeating the committed protocol with seed sweeps and confidence intervals would convert the point estimates into bounded estimates and would resolve whether the negative-ΔCatch ablation entries are stable (Section 6.1.3).

Close the semantic-class gap. The three meaning-dependent flaw classes (contradictory, out-of-scope-target, ambiguous) remain caught at 40–59% because no deterministic signal exists for them (Section 6.1.4). Structured prompting that forces the model to enumerate decision grounds, targeted enumerators for scope, or a bounded-sampling escalator on the agreement gate are concrete candidate directions within the offline budget (Section 2.3).

Complete the production validation. The C1–C2 happy-path re-run against the production `gemma4:e4b` configuration is recorded as a TODO in Section 6.4 until the model is provisioned on the evaluation machine. Latency, throughput and energy profiling on the target hardware were not measured and are required before interactive-use claims (Section 7.5).

Close the engineering configuration gaps. Shipping hardening requires removing the default development API key, wiring the `production` scope-mode value to the `BLOCK` path of the enforcement policy, and reconciling the doc-only dependencies (spaCy, jsonschema runtime, LoRA, Mahalanobis and XAI feature-flag paths) with their documented roles (Section 7.4).

Extend the evaluation surface. The threat model closes at the retrieval-free, no-free-form-output boundary (Section 3.6). Future work could expand the boundary to indirect prompt injection, device-compromise and backdoored-weights scenarios, or relax the deployment constraint to measure what a fine-tuned backend (the planned LoRA Phase 2 work) adds to the safety record.

Re-verify the commercial position. The comparative assessment cites no named commercial product for lack of verifiable primary sources (Section 2.4); a commercialisation step should survey named products with primary sources and re-verify the gap claim before any positioning statement is made (Section 8.2).

REFERENCE LIST

[1] G. Deng et al., "Pentestgpt: Evaluating and harnessing large language models for automated penetration testing," in *33rd USENIX Security Symposium (USENIX Security 24)*, Philadelphia, PA: USENIX Association, 2024, pp. 847–864. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/deng

[2] A. Happe and J. Cito, "Getting pwned by ai: Penetration testing with large language models," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 2082–2086. DOI: 10.1145/3611643.3613083

[3] Z. Ji et al., "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023. DOI: 10.1145/3571730

[4] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, 2023, pp. 79–90. DOI: 10.1145/3605764.3623985

[5] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *33rd USENIX Security Symposium (USENIX Security 24)*, Philadelphia, PA: USENIX Association, 2024, pp. 1831–1847. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei

[6] OWASP Foundation, *Owasp top 10 for large language model applications*, 2025. [Online]. Available: https://genai.owasp.org/resource/owasp-top-10-for-llm-applications-2025

[7] P. Manakul, A. Liusie, and M. J. Gales, "Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 9004–9017. DOI: 10.18653/v1/2023.emnlp-main.557

[8] L. Kuhn, Y. Gal, and S. Farquhar, "Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation," in *Proceedings of the 11th International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=VD-AYtP0dve

[9] S. Farquhar, J. Kossen, L. Kuhn, and Y. Gal, "Detecting hallucinations in large language models using semantic entropy," *Nature*, vol. 630, no. 8017, pp. 625–630, 2024. DOI: 10.1038/s41586-024-07421-0

[10] X. Wang et al., “Self-consistency improves chain of thought reasoning in language models,” in *Proceedings of the 11th International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=1PL1NIMMrw

[11] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive nlp tasks,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.

[12] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using siamese bert-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992. DOI: 10.18653/v1/D19-1410

[13] G. F. Lyon, *Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Security Scanning*. Insecure.com LLC, 2009.

[14] Rapid7, *Metasploit framework*, 2024. [Online]. Available: https://www.metasploit.com/

[15] Y. Rekhter, R. Moskowitz, D. Karrenberg, G. J. de Groot, and E. Lear, “Address allocation for private internets,” IETF, Tech. Rep. RFC 1918, 1996. [Online]. Available: https://www.rfc-editor.org/rfc/rfc1918

[16] D. Kang, X. Li, I. Stoica, M. Zaharia, C. Guestrin, and T. Hashimoto, “Exploiting programmatic behavior of llms: Dual-use through standard security attacks,” in *2024 IEEE Security and Privacy Workshops (SPW)*, San Francisco, CA: IEEE, 2024, pp. 132–143. DOI: 10.1109/SPW63631.2024.00018

[17] National Institute of Standards and Technology, *National vulnerability database (nvd)*. [Online]. Available: https://nvd.nist.gov/

[18] FIRST.org, *Exploit prediction scoring system (epss)*. [Online]. Available: https://www.first.org/epss/

GLOSSARY

Engagement scope — The authorised network range and policy boundaries within which a security task may legitimately be executed.

Guard stage — A deterministic check applied around model inference to validate or reject its output before it is passed onward.

Intent contract — The structured, machine-readable representation of a parsed security command (intent type, target, recognised options and validation findings) produced by the Intent Recognition Engine.

[TODO: Extend the glossary with any further project-specific terms introduced by later chapters.]

APPENDIX A: RESEARCH INSTRUMENTS OR EXTENDED RESULTS

Introduction to this appendix

[TODO: Appendix content not yet generated. Reserved for supplementary material: extended result tables, experiment instrumentation, dataset composition detail, configuration listings, or selected code that is too long for the body. Reference only artefacts that exist (committed experiment artifacts, source listings).]

APPENDIX B: ACADEMIC INTEGRITY AND AI USE DISCLOSURE

[TODO: Appendix B (Academic Integrity and AI Use Disclosure) not yet drafted. When produced it must accurately describe the AI tools used, their purpose, the level of assistance, and the candidate's human verification and responsibility for the final content. Institutional policy wording must be obtained from the official source, not fabricated (master rule 35/appendix B scope).]